Predicting US Violent Crime Rates

Table of Contents

1. INTRODUCTION **1**

2. METHODOLOGY **2**

2.1 Data Preparation 2
2.2 Singular Value Decomposition (SVD) 2
2.3 Exploratory Data Analysis 2
2.4 Model Optimisation 3
2.5 Markov Decision Process 4

3. RESULTS **6**

3.1 Data Preparation 6
3.2 SVD 7
3.3 Exploratory Data Analysis 7
3.4 Model Optimisation 12
3.5 Markov Decision Process 18

4. DISCUSSION **19**

4.1 Interpretation of Findings and Technical Impact 19
4.2 Ethical Considerations and Mitigation Strategies 19

5. CONCLUSIONS **21**

6. REFERENCES **21**

APPENDICES **22**

Appendix A – Variable Descriptions 22
Appendix B – Supporting Visualisations 23
Appendix C – Full Grid Search Implementation 25
Appendix D – Full MDP Definition 5
Appendix E – Code 26

1. Introduction

This project applies linear algebra, calculus, statistics, and probability to violent crime prediction. Using the *Communities and Crime* dataset, the objective is twofold: to engineer a robust predictive model for crime rates and to formulate an optimal policy for law enforcement resource allocation.

First, Singular Value Decomposition (SVD) is employed to address the curse of dimensionality inherent in the high-dimensional feature space. Second, Exploratory Data Analysis (EDA) utilises hypothesis testing to assess the complex correlations between socio-economic disadvantage and crime. Third, a Ridge Regression model is

optimised using Gradient Descent, demonstrating the role of calculus in loss minimisation and hyperparameter optimisation. Finally, the problem is reframed as a Markov Decision Process (MDP) to calculate an optimal policy (π^*) that balances immediate enforcement costs against long-term societal benefits.

The report concludes with a critical examination of AI ethics.

2. Methodology

2.1 Data Preparation

The *Communities and Crime* dataset was sourced from the UCI repository[1]. It contains data on 1,994 communities in the US with 127 features describing the demographic, economic and social conditions of those communities and a target variable `ViolentCrimesPerPop` measuring the per capita violent crime rate.

Initial inspection revealed missing values encoded as '?' and features with > 80% missing data were removed. For the remaining missing values (specifically `OtherPerCap`), mean imputation was applied for the respective state. To prepare the data for SVD, the categorical variable `communityname` was transformed using One-Hot Encoding (OHE). Finally, all numerical features were centred for SVD.

2.2 Singular Value Decomposition (SVD)

We applied Singular Value Decomposition (SVD) to the pre-processed feature matrix A[2]. SVD decomposes the matrix into three orthogonal components:

$$A = U\Sigma V^T$$

Where Σ is a diagonal matrix of singular values representing the variance captured by each component. We calculated the cumulative explained variance ratio by squaring the singular values and dividing by the total variance. We used this to select a reduced subset of components that retained >90% of the original information.

2.3 Exploratory Data Analysis

Univariate analysis assessed the target variable's distribution visually (histogram) and statistically (skewness, kurtosis). This allows us to assess the assumptions of normality inherent in the linear model and in t-tests.

We conducted bivariate analysis to identify the features that correlate most strongly with the target variable and each other. This allowed us to test the independence assumptions in the model. We assessed the correlations using both Pearson's correlation coefficient (r) against Spearman's rank correlation (ρ) to test the linearity assumptions in the model. Scatterplots were used to examine the relationships between continuous variables and boxplots were used to compare the distributions of

continuous variables grouped by categorical variables (binned continuous variables). The Variance Inflation Factor (VIF) was calculated to assess multicollinearity[3].

To formally test the relationship between socio-economic factors and crime, we formulated two specific hypotheses (H_1: Poverty, H_2: Race). Given the unequal variances observed between groups we utilised Welch's t-test to determine statistical significance ($p < 0.05$)[4].

2.4 Model Optimisation

We formulated the crime prediction task as a linear regression problem optimised via Gradient Descent[5]. To address the significant multicollinearity identified during EDA, we applied Ridge Regression (L2 Regularisation)[6]. This adds a penalty term to the cost function, constraining the magnitude of the model weights (θ) to reduce overfitting.

The objective function (Cost) $J(\theta)$ is defined as:

$$J(\theta) = \frac{1}{m}||X\theta - y||^2 + \lambda||\theta||^2$$

Where λ is the regularisation hyperparameter. To minimise this cost, we derived the gradient vector $\nabla J(\theta)$ with respect to the parameters θ:

$$\nabla J(\theta) = \frac{2}{m}X^T(X\theta - y) + 2\lambda\theta$$

We implemented this algorithm using vectorised operations in NumPy (Figure 1).

The data was split into training, validation, and test sets (60%-20%-20%). A Grid Search strategy to identify the optimal learning rate and batch size through cross-validation. While random search for hyperparameter optimisation has been proven to be more efficient in high-dimensional spaces (typically neural networks), linear regression and gradient descent are low-dimensional and suitable for grid search[7].

Table 1: Hyperparameters Evaluated in Grid Search

Hyperparameter	**Values Included**
Learning Rate	0.0001, 0.01, 1, 1.5
Batch Size	32, 64, 1,196 (Full Training Set)

Figure 1: Gradient Descent with L2 Regularisation Implementation

```
def lin_reg_gradient(X_b, y, alpha, theta):
    m = len(y)
    gradients = (2/m) * X_b.T.dot(X_b.dot(theta) - y)
    if alpha > 0:
        reg_term = 2 * alpha * theta.copy()
        reg_term[0] = 0  # Zero out bias penalty
        gradients += reg_term
```

```
    return gradients
```

2.5 Markov Decision Process

To transition from prediction to decision support, we modelled the community policing problem as a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ)[8].

- **States (S):** A composite of Crime Level (High, Medium, Low) and Poverty Level (High, Low), resulting in 6 discrete states.
- **Actions (A):** The agent can choose between *Increase Patrols*, *Community Policing*, or *Do Nothing*.
- **Transition Probabilities (P):** We defined a transition matrix $T(s' \mid s, a)$ representing the probability of moving to a safer state given an action.
- **Rewards (R):** The reward function assigns positive values for low-crime states and negative values for high-crime states and intervention costs.

An essential element of the MDP is the *Markov Property*, which states that the probability of the next state depends only on the current state and action, not on the full history of proceeding states and actions.

Our objective here is to present a simplified, illustrative version so we deliberately keep the state space small, but a much richer state space that contains information about past actions is possible[9].

The State Transition Graph in Figure 3 illustrates the probabilities of transitioning between states (S0-S6) given an action taken (A0-A2), and the reward for taking that action in the state (R0-R5). (Figure 2)

The graph conveys three core ideas in our model:

1. Poverty moderates the effectiveness of police action on crime rates.
2. Police action does not influence poverty rates as it operates on a much shorter time horizon.
3. Lowering crime is the goal so every action in a low crime state has a higher reward than a medium crime state, and so on, encouraging transitions.

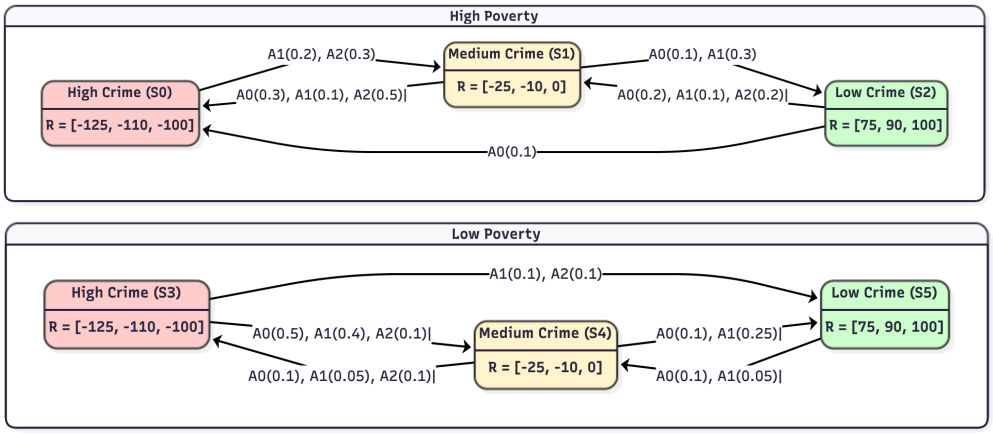
Figure 2: Full MDP Implementation

```python
# States
states = [
    "High Crime, High Poverty",       # S0
    "Medium Crime, High Poverty",     # S1
    "Low Crime, High Poverty",        # S2
    "High Crime, Low Poverty",        # S3
    "Medium Crime, Low Poverty",      # S4
    "Low Crime, Low Poverty"          # S5
]

# Actions
actions = [
    "Increase Patrols",               # A0
    "Increase Community Policing",    # A1
    "Do Nothing"                      # A2
]

num_states = len(states)
num_actions = len(actions)
```

```python
# --- REWARDS (R) ---
# R(s,a) = State Value - Action Cost
# State Values: High=-100, Med=0, Low=100
# Action Costs: Patrol=25, Community=10, Nothing=0

# High Poverty Rewards
R[0] = [-125, -110, -100]    # State 0 (High Crime)
R[1] = [-25, -10, 0]         # State 1 (Medium Crime)
R[2] = [75, 90, 100]         # State 2 (Low Crime)

# Low Poverty Rewards (Symmetric values)
R[3] = [-125, -110, -100]    # State 3 (High Crime)
R[4] = [-25, -10, 0]         # State 4 (Medium Crime)
R[5] = [75, 90, 100]         # State 5 (Low Crime)
```

```python
# --- TRANSITIONS (T) ---

# --- HIGH POVERTY (Hard Mode) ---
# S0: High Crime. Sticky. Patrols struggle.
T[0, 0] = [0.8, 0.2, 0.0, 0, 0, 0] # Patrol
T[1, 0] = [0.7, 0.3, 0.0, 0, 0, 0] # Community
T[2, 0] = [1.0, 0.0, 0.0, 0, 0, 0] # Nothing

# S1: Med Crime. Volatile. Patrols cause backlash.
T[0, 1] = [0.3, 0.6, 0.1, 0, 0, 0] # Patrol (Risk of regression)
T[1, 1] = [0.1, 0.6, 0.3, 0, 0, 0] # Community
T[2, 1] = [0.5, 0.5, 0.0, 0, 0, 0] # Nothing (Fast regression)

# S2: Low Crime. Maintenance required.
T[0, 2] = [0.1, 0.1, 0.8, 0, 0, 0] # Patrol (Small %, big regression)
T[1, 2] = [0.0, 0.1, 0.9, 0, 0, 0] # Community
T[2, 2] = [0.0, 0.2, 0.8, 0, 0, 0] # Nothing

# --- LOW POVERTY (Easy Mode) ---
# S3: High Crime. Responsive to force.
T[0, 3] = [0, 0, 0, 0.4, 0.5, 0.1] # Patrol (Works well)
T[1, 3] = [0, 0, 0, 0.5, 0.4, 0.1] # Community
T[2, 3] = [0, 0, 0, 0.9, 0.1, 0.0] # Nothing

# S4: Med Crime. Resilient.
T[0, 4] = [0, 0, 0, 0.1, 0.3, 0.6] # Patrol
T[1, 4] = [0, 0, 0, 0.05, 0.45, 0.5] # Community
T[2, 4] = [0, 0, 0, 0.1, 0.9, 0.0] # Nothing

# S5: Low Crime. Stable.
T[0, 5] = [0, 0, 0, 0, 0.9, 0.1] # Patrol (Small regression risk)
T[1, 5] = [0, 0, 0, 0, 0.0, 1.0] # Community (Perfectly stable)
T[2, 5] = [0, 0, 0, 0, 0.05, 0.95] # Nothing (Small drift risk)
```

Figure 3: State Transition Graph (with High Poverty and Low Poverty Sub-Graphs)



We observe the relationship between poverty and crime and the interaction effects that arise in complex social outcomes (e.g. between poverty, race and crime) in the EDA section for this dataset. This motivated our selection of poverty as a component of the state description. We can estimate the probability of being in these states using by grouping the values of `ViolentCrimesPerPop` and `PctPopUnderPov` into 3 bins.

Table 2: % of Communities in 3-Binned Crime and Poverty Categories

Category (Binned, Normalised Values)	Crime	Poverty
Low (0.0 – 0.3)	61%	76%
Medium (0.3 – 0.7)	31%	17%
High (0.7 – 1.0)	8%	7%

However, this dataset is cross-sectional rather than longitudinal, so it provides no indication of the probability of *moving between* states. The probability and reward values are therefore estimated to simulate the effect of policy choices and to illustrate the mechanics of the MDP. Reverse reinforcement learning could be used in future work to more robustly estimate the reward function and transition probabilities[10].

We applied the Value Iteration algorithm to compute the optimal policy $\pi^*(s)$ that maximises the expected discounted cumulative reward, solving the Bellman Optimality Equation[8]:

$$V^*(s) = \max_{a \in A} \sum_{s'} T(s' \mid s, a)\, [R(s, a, s') + \gamma V^*(s')]$$

3. Results

3.1 Data Preparation

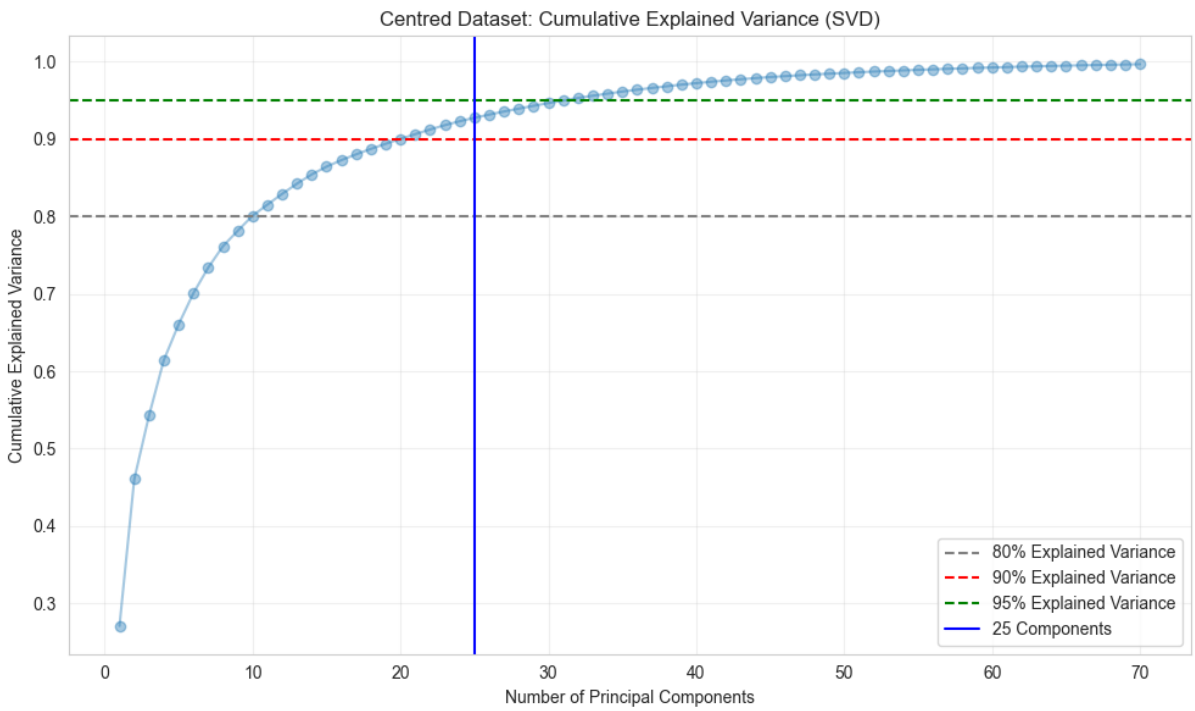
The initial data inspection revealed a dataset of 1,994 instances and 127 features. After removing features with excessive missing values (primarily LEMAS survey data) and

imputing the state mean for `OtherPerCap`, the final processed dataset contained 99 numeric features. Three datasets were created – `X_base`, `X_centred` and `X_centred_and_ohc` – to illustrate the impact of the pre-processing techniques (centring and one-hot encoding) on dimensionality reduction and prediction.

3.2 SVD

The SVD analysis demonstrated significant redundancy in the feature space. The first principal component alone captured ~25% of the variance in the centred data. The cumulative explained variance ratio indicated that 25 components were sufficient to retain $> 90\%$ of the dataset's information, validating the use of dimensionality reduction to reduce computational overhead without significant information loss (Figure 4). The non-centred data contained most of the variance in PC1 as it was simply reflecting the mean[2]. One-hot encoding of `communityname` added no new information while adding significant dimensionality.

Figure 4: *Cumulative Explained Variance of Principal Components for* `X_centred`



3.3 Exploratory Data Analysis

The target variable, `ViolentCrimesPerPop`, is right-skewed (Skewness =1.5,) and leptokurtic (Kurtosis = 1.8). While most communities have a low violent crime rate (median = 0.15), 44 communities have a very high violent crime rate (1). (Figure 5)

The formal assumption of the linear regression model is that that the residuals have a normal distribution. The non-normal distribution of the target variable indicates this

may not hold true and necessitates a formal test of the residual distribution after the model has been built[11].

Correlation analysis identified reasonably strong linear relationships between violent crime and race, poverty, household composition and education. The 10 strongest positive and negative correlations are illustrated in Table 3. The descriptions for these variables are in Appendix A.

Table 3: 10 Features with Strongest Positive and Negative Correlations to Violent Crime

Top 10 Positive Correlations			**Top 10 Negative Correlations**	
Feature	**Pearson's *r***		**Feature**	**Pearson's *r***
PctIlleg	0.74		PctKids2Par	-0.74
racepctblack	0.63		PctFam2Par	-0.71
pctWPubAsst	0.57		racePctWhite	-0.68
FemalePctDiv	0.56		PctYoungKids2Par	-0.67
TotalPctDiv	0.55		PctTeen2Par	-0.66
MalePctDivorce	0.53		pctWInvInc	-0.58
PctPopUnderPov	0.52		PctPersOwnOccup	-0.53
PctUnemployed	0.50		PctHousOwnOcc	-0.47
PctHousNoPhone	0.49		medFamInc	-0.44
PctNotHSGrad	0.48		medIncome	-0.42

The problem of multicollinearity immediately becomes apparent through the form of multiple iterations of the same feature e.g. race (`racepctblack`, `racePctWhite`), kids in the household (`PctKids2Par`, `PctFam2Par`, `PctYoungKids2Par`, `PctTeen2par`) all featuring among the most correlated variables.

One option at this stage is to choose a representative feature from each category (e.g. `racePctWhite` and `PctKids2Par` as they have the strongest correlations) and remove all others from the dataset. However, even among distinct feature categories there are significant correlations between features. The correlation matrix (Figure 6) shows that communities with high rates of poverty also have high rates of unemployment and lower proportions of white people, for example.

To empirically quantify the extent of multicollinearity, we calculated VIF. A VIF of 10 is considered high[12] and there are 20 features with over 100(!) in this dataset. (Appendix B). This problem of multicollinearity directly informs our choice of Ridge Regression described in the methodology.

Along with Pearson's r we calculated Spearman's ρ to examine nonlinear relationships in the data. Most features show higher values for Spearman's ρ which indicate a degree of non-linearity in the relationship. (Appendix B)

For example, Spearman's *p* is 0.1 larger for `PctPopUnderPov`, and we see that the violent crime rate rises steadily until a poverty rate of 0.6 but then starts to level off in Figure 7. There is some uncertainty about the measurement in the latter categories as the sample size drops below 50 in the 0.8-0.9 and 0.9-10 bins, but the indication is that the relationship between crime and poverty is not perfectly linear.

We also observed a non-linear interaction effect: while poverty correlates with crime across all demographics, communities with high proportions of black people exhibited a steeper increase in crime rates as poverty levels rose.

This poses a problem for our linear model[12].

A set of independent samples t-tests were conducted to test two hypotheses about the relationship between crime and poverty, and crime and race. All tests were conducted at a significant level of $p < 0.05$.

Hypothesis testing confirmed these observations were statistically significant. Welch's t-test rejected the null hypothesis for both Poverty (H_1) and Race (H_2) with $p < 0.001$, confirming that socio-economic disparities are statistically inseparable from crime rates in this dataset.

Table 4: 10 Independent T- Test Results for Poverty (H_1) and Race (H_2)

	Poverty Rate (PctPopUnderPov)		**White Race % (racePctWhite)**	
	Mean	**Variance**	**Mean**	**Variance**
<= Median	0.12	0.01	0.36	0.06
> Median	0.36	0.07	0.11	0.01
t-statistic	25.98		-29.19	
p-value	2.8e-120 ($p < 0.001$)		8.1e-147 ($p < 0.001$)	
D of F	1,305		1,406	

The t-test requires a normal distribution of sample means. While our target variable has a non-normal distribution, the Central Limit Theorem (CLT) tells us that at a large enough sample size the distribution of the means from those samples will be approximately normal[11]. Our sample size is large (1,994) and the distribution of the sample means does adhere to the normality assumption, validating the t-test results. (Appendix B)

Figure 5: *Distribution of Violent Crime Rate (Histogram and Kernel Density Estimation)*

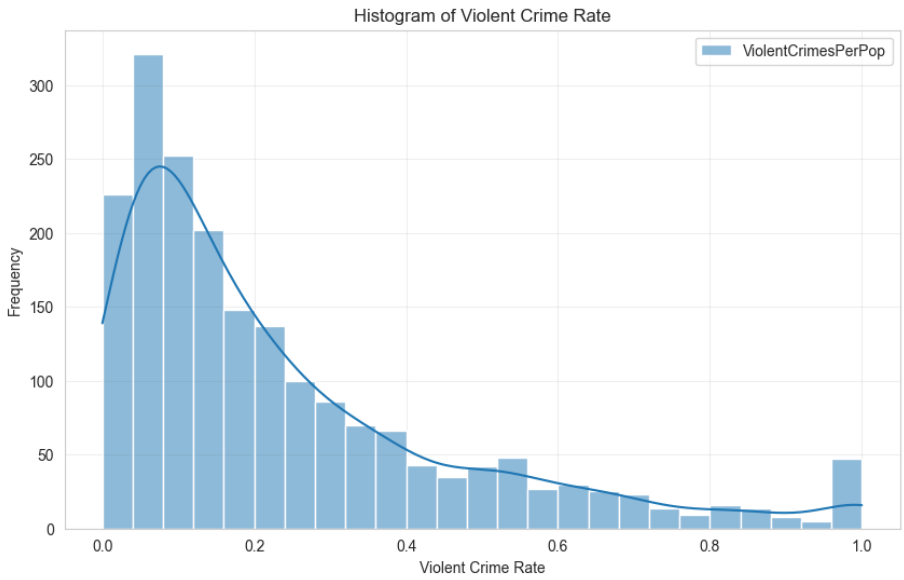


Figure 6: Correlation Heatmap of Key Features

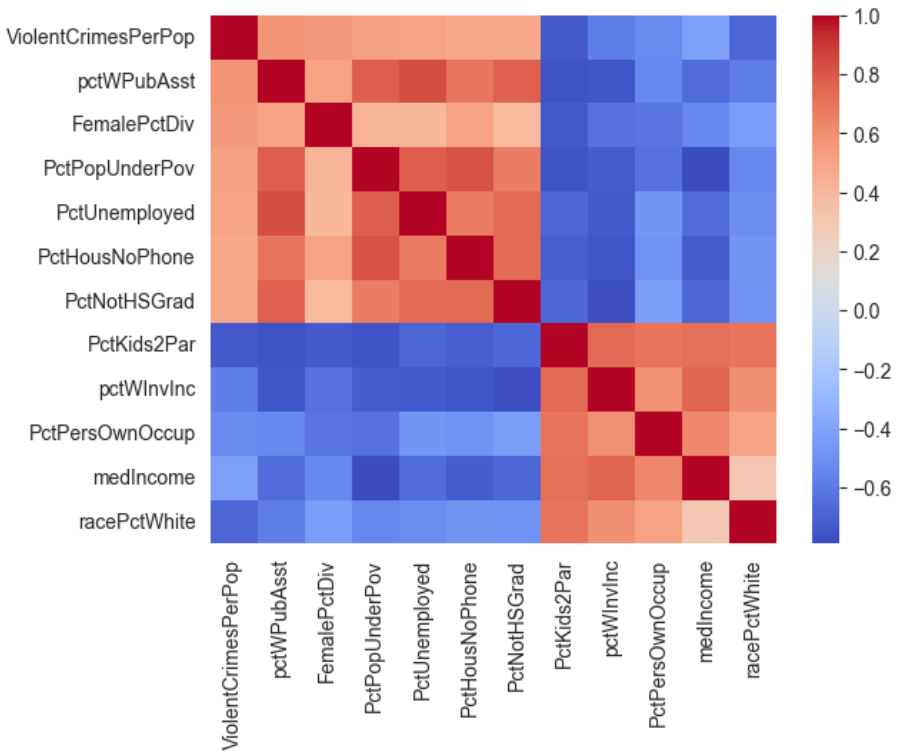


Figure 7: *Interaction Effects Between Poverty and Race on Violent Crime*

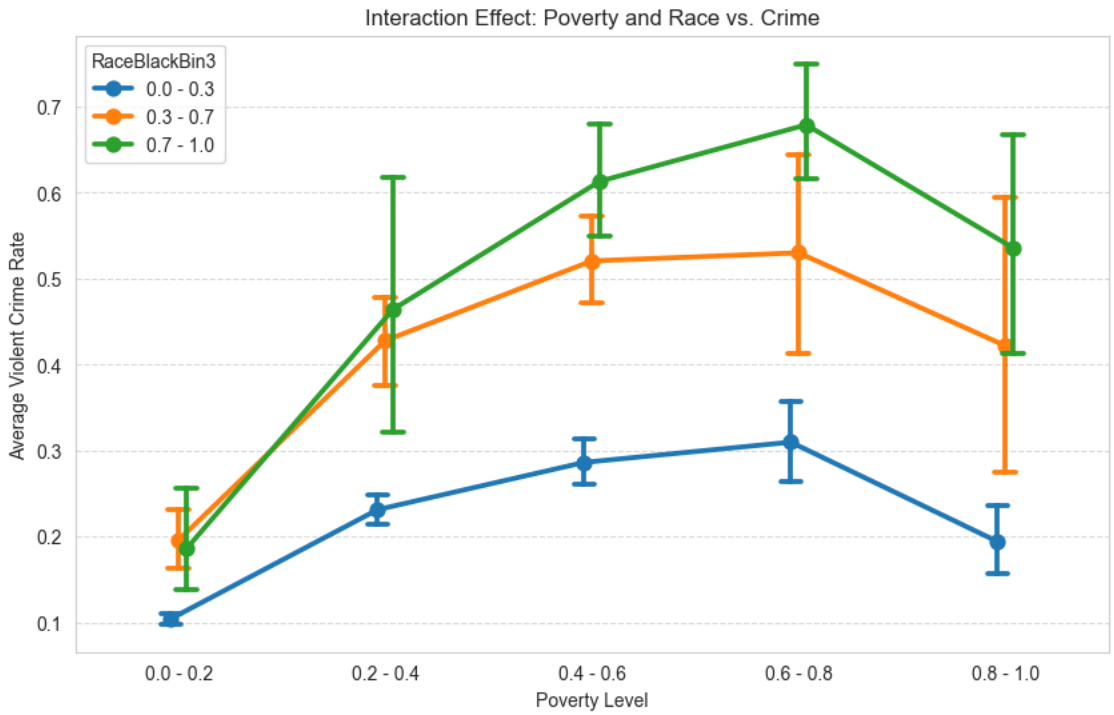
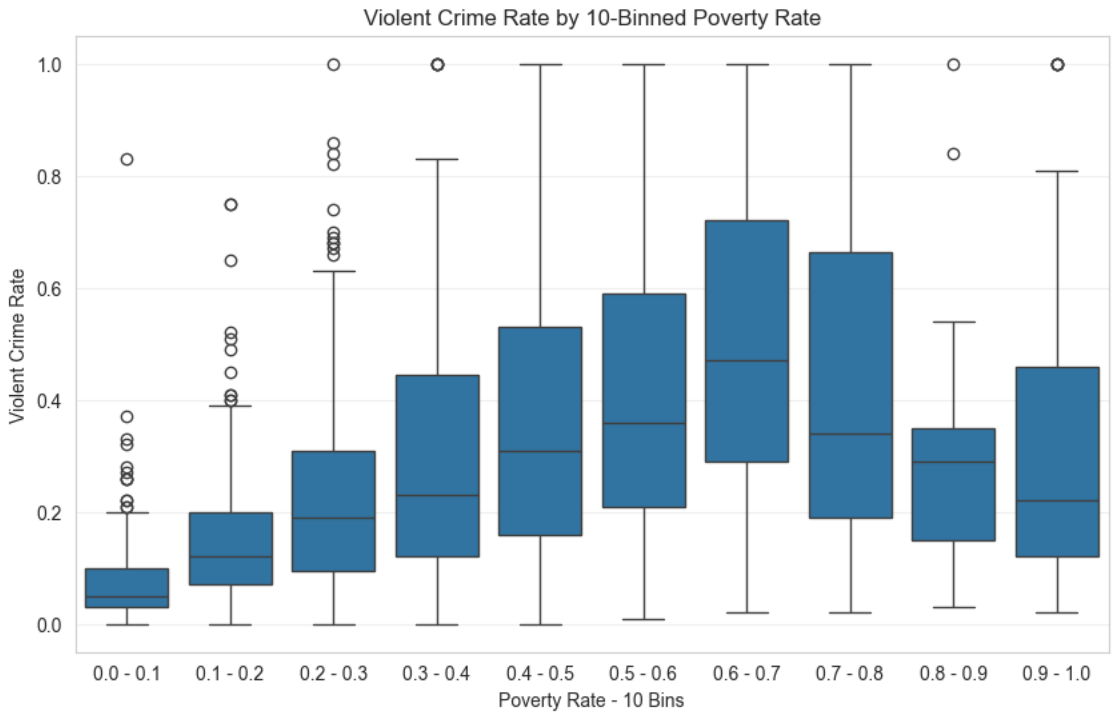


Figure 8: Boxplot of Violent Crime Rate by 10-Binned Poverty Rate (with sample sizes)



n=	470	410	271	238	226	165	89	56	29	40

3.4 Model Optimisation

The Ridge Regression model was optimised using Mini-Batch Gradient Descent. The training dynamics (Figure 9) demonstrate the effects of the hyperparameter tuning. The key findings were:

- **Optimisation**: The Grid Search identified an optimal initial learning rate of 1 and a Batch Size of 32. (Figure 10)
- **Multicollinearity**: Ridge successfully handled multicollinearity by penalising large weights which improved model stability and interpretability.
- **Racial Bias**: Analysis of the learned weights θ revealed that `racepctblack` was the most important feature. Error rates varied significantly across racial groups.
- **Normality**: The residuals were approximately normally distributed (Shapiro-Wilk $p < 0.05$), validating the linear model assumptions (Figure 16).

Table 5: Hyperparameter Optimisation Results

Run	Learning Rate	Batch Size	Epochs Run	Validation MSE
0	0.0001	32	2000	0.0229
1	0.0001	64	2000	0.0245
2	0.0001	Full	2000	1.029
3	0.01	32	1139	0.0225
4	0.01	64	1150	0.0227
5	0.01	Full	1913	0.0228
6	1	32	1564	0.0212
7	1	64	1584	0.0215
8	1	Full	1560	0.0226
9	1.5	32	1607	0.0216
10	1.5	64	1594	0.0214
11	1.5	Full	1597	0.0224

The momentum, patience and decay hyperparameters define a "learning schedule"[5], dynamically adjusting the learning rate based on the steepness of the gradient and the rate of improvement in the validation MSE.

An initial round of hyperparameter tuning was conducted to identify the optimal hyperparameters with momentum (0.7), patience (10), decay (0.9) as the optimal values to fix the learning schedule. $\alpha = 0.1$ was also identified as the optimal hyperparameter for the regularisation term.

Figure 9 illustrates the training dynamics of different batch sizes and learning rates. At learning rates of 0.001 the learning curve is slow and gradual, but Batch Gradient Descent fails to converge. At learning rates of 1.5 the mini batches showed signs of instability, with MSE rising after a few iterations.

Figure 10 illustrates the training dynamics of the optimal hyperparameters (LR=1, BS=32). A learning rate of 1 allows the model to quickly find parameters that substantially reduce the MSE, but the stochasticity of the small batches at that learning rate produce noisy results. As the learning rate decays the loss curve becomes much smoother until it settles at an optimal state.

Using the optimal hyperparameters, we trained the final model on the combined training and cross-validation dataset and evaluated it on the hold-out test set. The test MSE is 0.021 vs. the training MSE of 0.020 showed it generalised well.

We evaluated the final model against two closed-form solutions: the normal equation and SVD. The normal equation $\theta = (X^T X)^{-1} X^T y$ fails due to a singular matrix: perfect correlation between two features.

SVD is more numerically stable and successfully handled a singular matrix. It produced a marginally better test MSE (0.19 vs. 0.20 for Ridge), however SVD showed signs of overfitting by producing large conflicting weights (e.g. +0.49 for `MalePctDivorce` and -0.62 for `TotalPctDiv`). (Figure 12) This harms model interpretability and makes it difficult to explain to stakeholders what e.g. the effect of an increase in the divorce rate would be.

Ridge produced smaller weights (c. -0.075 to +0.075) which makes the model more robust and the resulting feature weights are more interpretable (Figure 13). Overall, the results are highly correlated (r=0.96) (Figure 14), and Ridge sacrifices a small amount of accuracy for model interpretability; a key priority when considering AI ethics.

Analysis of the model weights confirms that many of the features identified in the correlation analysis – race, poverty, divorce rates – play an important role in the model. However, the model displays evidence of racial bias with median absolute error rates for communities with high levels of black people at 2x the level of those with low levels of black people. (Figure 15)

The normality assumption of the linear regression model was confirmed through the approximately normal distribution of the residuals displayed in the histogram (Figure 16) and through the Shapiro-Wilk test results ($w = 0.93, p < 0.01$).

Figure 9: Loss Curves by Batch Size and Learning Rate (log scale, shared y-axes per row)

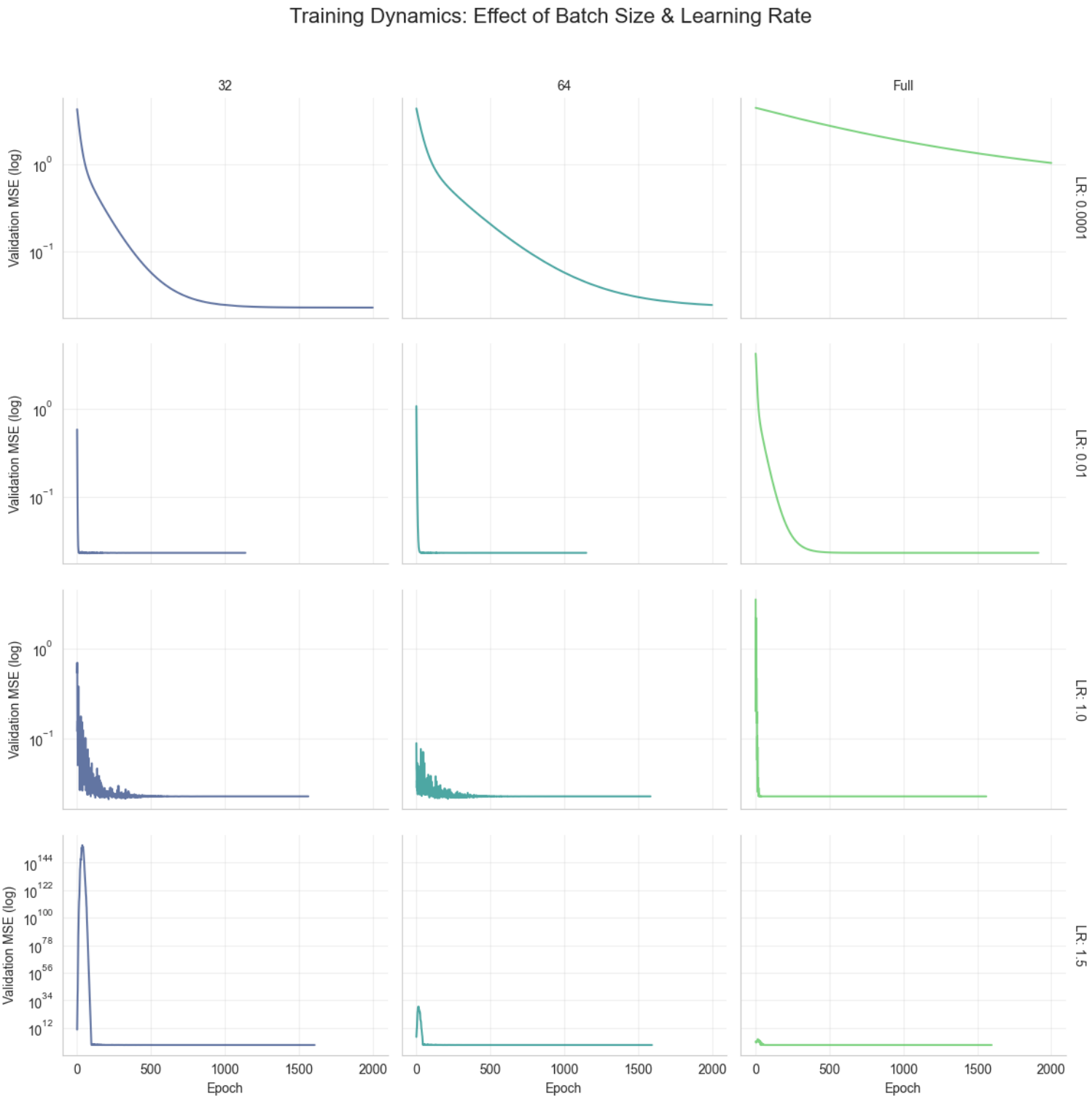


Figure 10: Loss Curve for Optimal Hyperparameters with Leaning Rate Decay

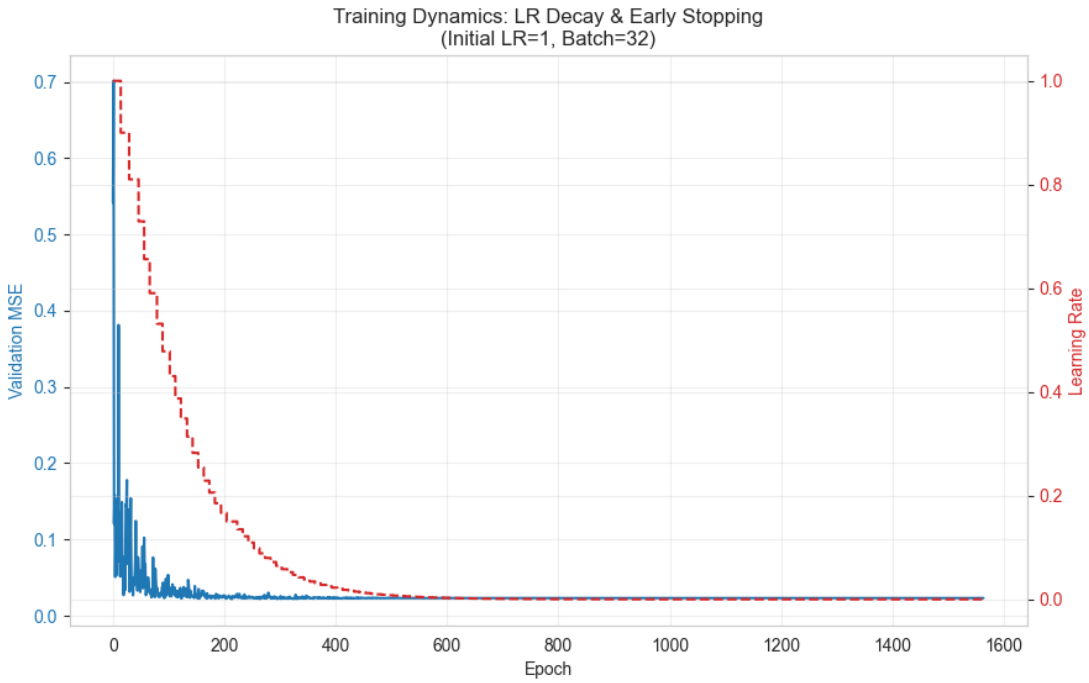
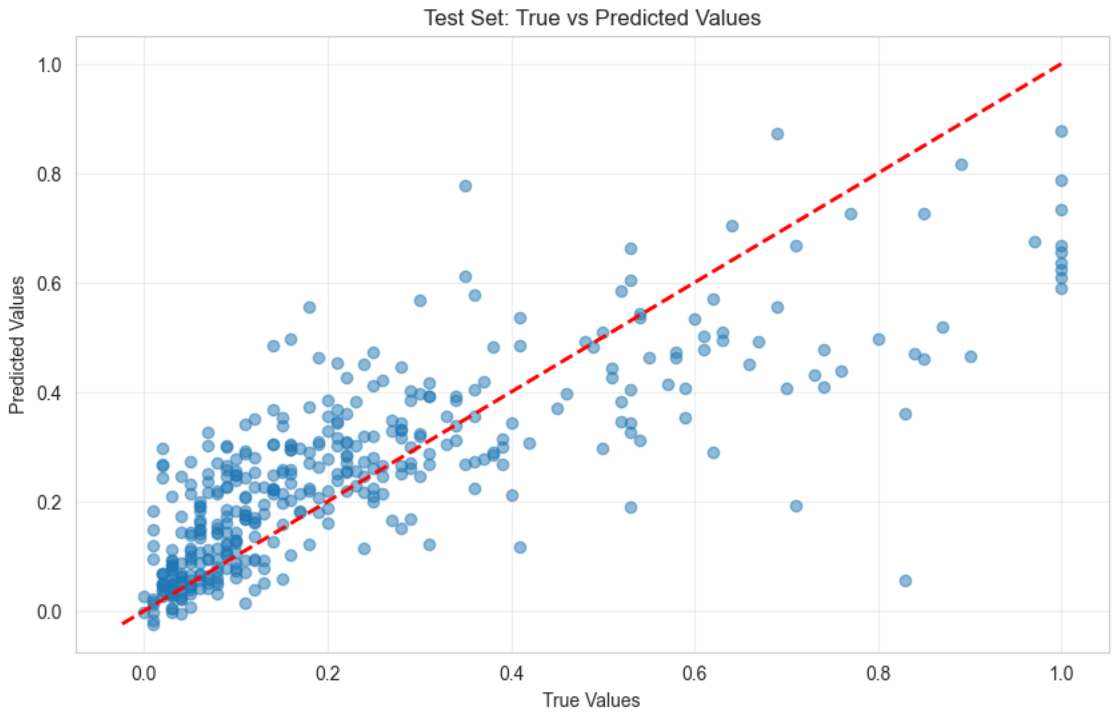


Figure 11: Scatterplot of True vs. Predicted Values on the Test Set




```
Training final model with best hyperparameters:
  Learning rate: 1
  Batch size: 32
Early stopping triggered at epoch 1595

Final model parameters (first 5 values): [ 0.24963668  0.02309094 -0.00285132  0.09072477 -0.07674668]
Final training MSE: 0.020
Test MSE: 0.021
```

Figure 12: Model Coefficients for Top / Bottom 10 Features for SVD

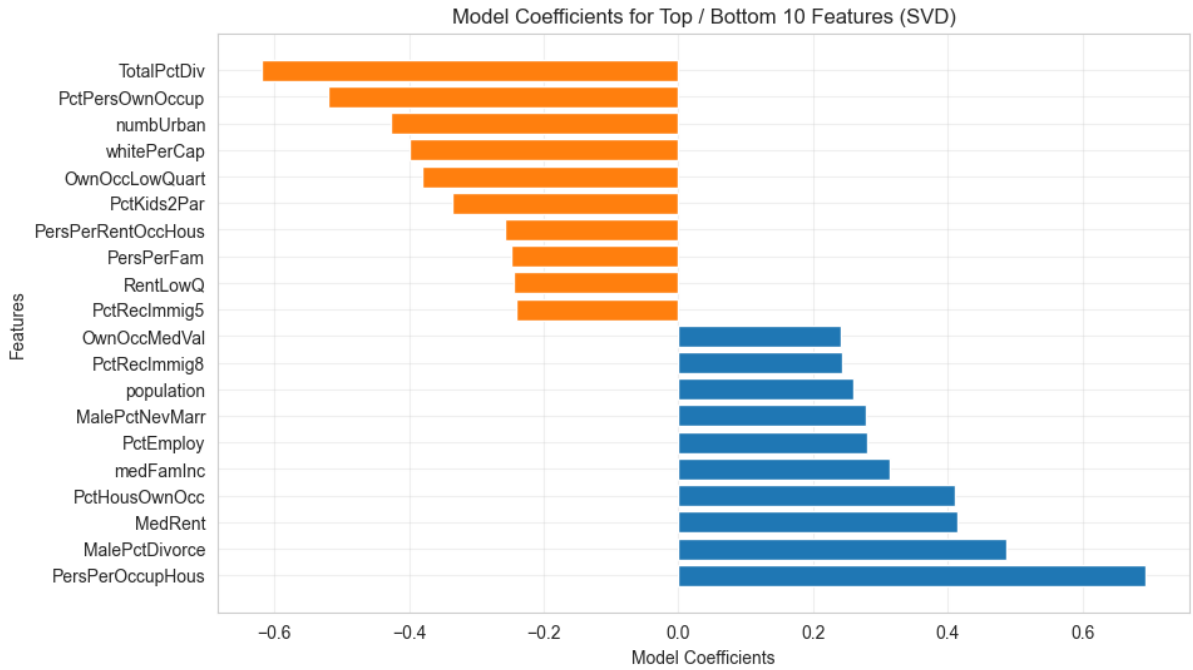


Figure 13: Model Coefficients for Top / Bottom 10 Features for Ridge

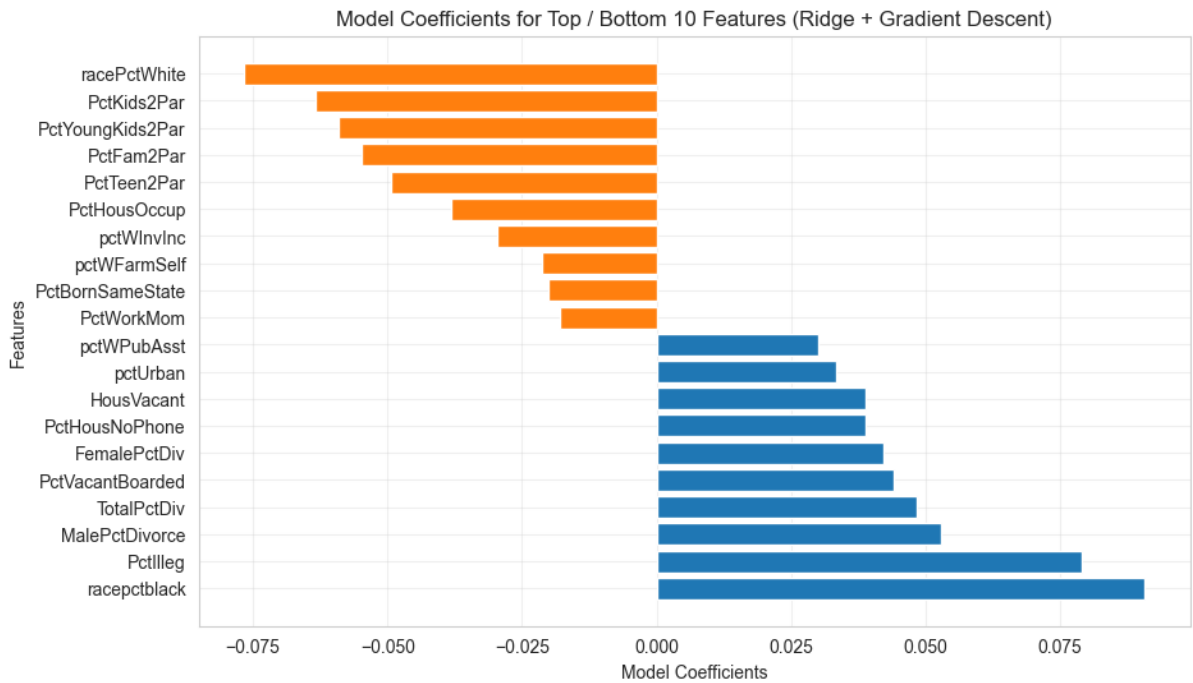


Figure 14: Scatterplot of Gradient Descent vs. SVD and Pseudo-Inverse Predictions

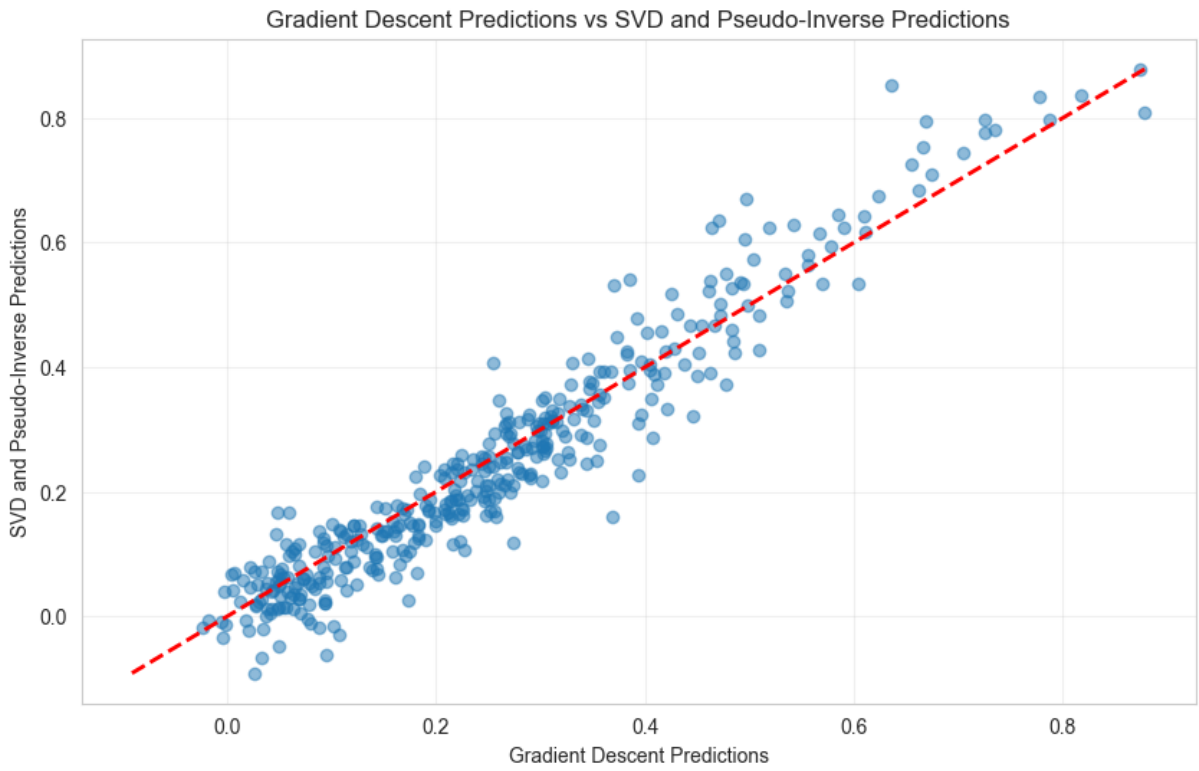


Figure 15: Absolute Error by Race (3-Binned `racepctblack`*)*

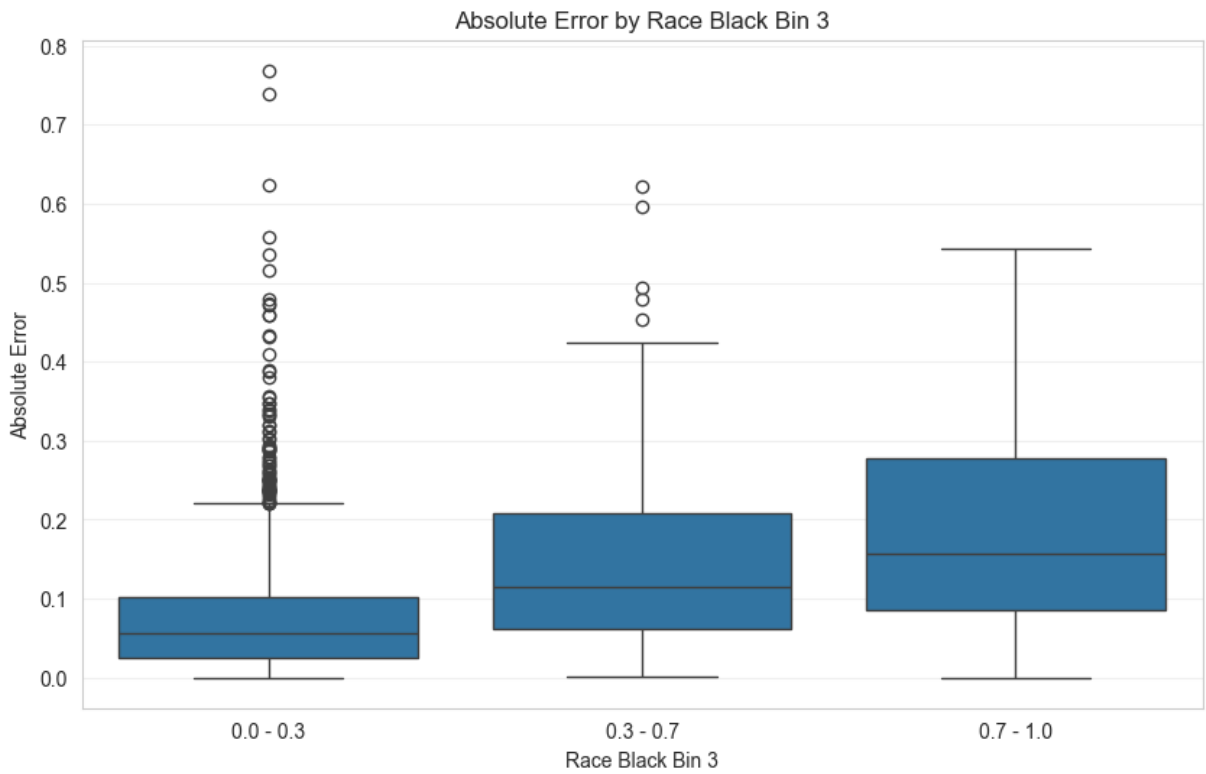


Figure 16: Distribution of Residuals on the Training Set

3.5 Markov Decision Process

The Value Iteration algorithm converged to a stationary optimal policy π^* that varied significantly based on the discount factor γ.

- **Myopic Policy ($\gamma = 0.05$):** The agent purely sought to minimise costs, choosing to do nothing in all states as it was the cheapest choice rather than trying to transition to more valuable states.
- **Patient Policy ($\gamma = 0.95$):** The agent prioritised long-term state transitions. As detailed in Table 1, the optimal policy shifted to Community Policing for all states except High Crime, Low Poverty where Increase Patrols brings a greater chance of transitioning to Medium Crime.

Table 6: Optimal policy π^* generated by *Value Iteration* $(\gamma = 0.95)$

State	**Action**
High Crime, High Poverty	Increase Community Policing
Medium Crime, High Poverty	Increase Community Policing
Low Crime, High Poverty	Increase Community Policing
High Crime, Low Poverty	Increase Patrols
Medium Crime, Low Poverty	Increase Community Policing
Low Crime, Low Poverty	Increase Community Policing

4. Discussion

4.1 Interpretation of Findings and Technical Impact

The results of this report suggest that violent crime is a systemic issue driven by socio-economic disadvantage rather than isolated individual choices. The EDA phase revealed that poverty (H_1) and racial segregation (H_2) are statistically inseparable from crime rates ($p<0.001$), creating a complex web of multicollinearity.

Building a model to predict crime rates using complex socio-economic factors raises two key trade-offs to navigate: bias-variance and accuracy-interpretability.

We used a linear regression model to predict the violent crime rate because they are "white box" models that are highly interpretable and allow you to transparently identify the impact of each feature in the dataset. This is a huge benefit over black box models (e.g. neural networks) in domains like predictive policing which require fair, transparent and accountable application of AI[14]. This motivates our choice of linear regression.

The downside of parametric models like linear regression is that they function on a set of strict, underlying statistical assumptions: linearity, homoscedasticity (constant variance), normality and independence. EDA allowed us to test these underlying assumptions[15].

While some linearity was identified, our assessment is the linear coefficients capture a significant portion of the relationship between the features and target, and the added benefit of interpretability makes it a valid choice in this domain vs e.g. neural networks that would be more capable of capturing the nonlinear relationships.

A closed model using SVD produced the lowest MSE at the cost of high variance caused by multicollinearity. Ridge Regression was designed to deal with this problem; introducing a small amount of bias to significantly reduce variance and produce more reasonable coefficients[6]. This final model produced a robust generalisation error (Test MSE = 0.02) with significant improvements in interpretability.

Using the MDP, the divergence between the Myopic Policy ($\gamma = 0.05$) and the Patient Policy ($\gamma = 0.95$) mathematically quantifies the cost of short-termism in public policy. The model illustrates that while aggressive policing can yield immediate rewards at lower costs, sustainable community investment optimises the cumulative reward by altering the transition probabilities of the environment itself.

4.2 Ethical Considerations and Mitigation Strategies

The predictive power of our model comes at a significant ethical cost. The heavy weighting of the `racepctblack` feature (Figure 13) and the compounding interaction effects with poverty highlight a critical risk of *Allocative Harm*[16]. A linear model trained

on this historical data will inevitably perpetuate, and potentially amplify, existing racial biases.

Police have historically patrolled low-income, minority communities more heavily than white, affluent ones and this introduces bias in the data[17]. Algorithms naively trained on this data will identify these communities as high risk and recommend police forces deploy in these "hotspots". With a larger force there, more crime data is collected, the model's predictions are validated, and the initial bias in the data is reinforced. This "runaway feedback loop" was empirically demonstrated by the EU's Fundamental Rights Agency (FRA)[18]. Risks are even greater for active optimisation through MDPs, where reward hacking can produce perverse outcomes from simple reward functions[19]. We propose a few mitigations to address this.

- **Fairness vs. Accuracy**: Removing sensitive features (like Race) often degrades model accuracy (MSE) because these features act as proxies for unobserved systemic inequities. To mitigate this, we recommend implementing *Equalised Odds* as a constraint during model training, ensuring that the False Positive Rate (incorrectly classifying a low-crime community as high-risk) is consistent across demographic groups.
- **Transparency**: We selected Linear Regression over non-linear "black box" models to ensure interpretability[14]. For more complex models, *SHAP (SHapley Additive exPlanations)* values can help understand the local contribution of specific features to individual predictions.
- **Privacy**: The dataset is aggregated at a community level which mitigates many privacy risks. For more granular datasets it is recommended to implement *Differential Privacy* mechanisms, adding calculated noise to statistical queries to protect individual community anonymity without destroying aggregate utility.

However, these mitigation strategies have their limits. The controversy around the software COMPAS illustrated one of the core challenges in AI ethics. ProPublica determined that the software was biased because the False Positive Rate for black defendants was 2x white defendants[20]. Northpointe determined that their model was fair because a risk score of 7 meant the same probability of re-offending for both black and white defendants[21].

This illustrates the debate around the "Impossibility Theorem of Fairness".[22] When base rates are different for different populations, it is impossible to satisfy multiple to definitions of fairness simultaneously. Choosing a fairness metric is not a technical optimisation problem with a correct answer, it is a political and ethical choice about which type of error is more socially damaging.

5. Conclusions

This project integrated foundational mathematical principles from linear algebra, calculus, and probability to engineer a robust AI solution for crime prediction and policy optimisation. The technical implementation demonstrated that optimisation techniques – specifically Ridge Regression trained via Mini-Batch Gradient Descent – can effectively extract signal from high-dimensional, multicollinear data.

The project moved beyond pure prediction by using a Markov Decision Process to model the consequences of intervention. The finding that a high discount factor($\gamma = 0.95$) shifts the optimal policy from enforcement to investment provides a mathematical justification for sustainable community policing.

However, the ethical analysis confirms that accuracy is an inadequate metric for success in high-stakes domains. The strong dependence of the model on racial proxies necessitates a Human-in-the-Loop (HITL) governance framework. AI should serve as a decision-support tool to highlight systemic risks, not as an autonomous agent for deployment.

Word Count: 3,288 (excl. tables, figures, references and appendices)

6. References

1. Redmond & Baveja (2002). *A data-driven software tool for enabling cooperative information sharing among police departments*.
2. Brunton & Kutz (2019). *Data-Driven Science and Engineering*.
3. Penn State STAT 462. *Detecting Multicollinearity Using VIF*.
4. Delacre et al. (2017). *Why Psychologists Should use Welch's t-test*.
5. Ruder (2016). *An overview of gradient descent optimization algorithms*.
6. Hoerl & Kennard (1970). *Ridge Regression: Biased Estimation for Nonorthogonal Problems*.
7. Bergstra & Bengio (2012). *Random Search for Hyper-Parameter Optimization*.
8. Puterman (1994). *Markov Decision Processes*.
9. Baycik et al. (2020). *MDP approach for drug trafficking enforcement*.
10. Sutton & Barto (2018). *Reinforcement Learning*.
11. Hastie, Tibshirani, & Friedman (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*.
12. Marcoulides & Raykov (2019). *Evaluation of Variance Inflation Factors in Regression Models Using Latent Variable Modeling Methods*.
13. Grinstead, Charles & Snell. (1997). *Introduction to Probability.*
14. Rudin (2019). *Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead*.

15. Tukey (1977). *Exploratory Data Analysis*.
16. Barocas, Hardt & Narayanan. *Fairness and machine learning*.
17. Lum & Isaac (2016). *To Predict and Serve?*
18. EU FRA (2022). *Bias in algorithms - Artificial intelligence and discrimination*.
19. Henderson, Chugg, Anderson & Ho. *Beyond Ads: Sequential Decision-Making Algorithms in Law and Public Policy.*
20. ProPublica (2016). *Machine bias*.
21. Flores, Bechtel & Lowenkamp (2016). *False Positives, False Negatives, and False Analyses: A Rejoinder to "Machine Bias"*.
22. Bell, Bynum, Drushcak, Herasymova, Rosenblatt, Stoyanovich (2023). *The Possibility of Fairness: Revisiting the Impossibility Theorem in Practice*.

Appendices

Appendix A – Variable Descriptions

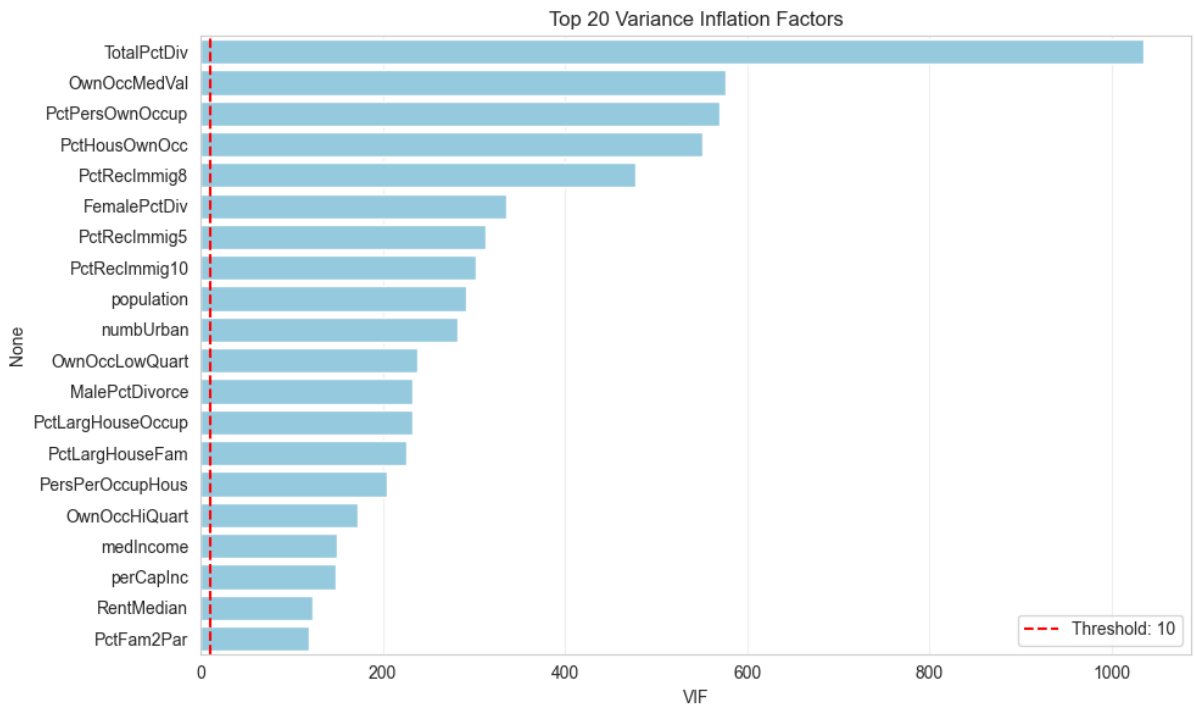
Variable	Category	Variable Info
racepctblack	Race	percentage of population that is african american (numeric - decimal)
racePctWhite	Race	percentage of population that is caucasian (numeric - decimal)
medIncome	Economics	median household income (numeric - decimal)
pctWInvInc	Economics	percentage of households with investment / rent income in 1989 (numeric - decimal)
pctWPubAsst	Economics	percentage of households with public assistance income in 1989 (numeric - decimal)
medFamInc	Economics	median family income (differs from household income for non-family households) (numeric - decimal)
PctPopUnderPov	Economics	percentage of people under the poverty level (numeric - decimal)
PctNotHSGrad	Education	percentage of people 25 and over that are not high school graduates (numeric - decimal)
PctUnemployed	Economics	percentage of people 16 and over, in the labor force, and unemployed (numeric - decimal)
MalePctDivorce	Household	percentage of males who are divorced (numeric - decimal)
FemalePctDiv	Household	percentage of females who are divorced (numeric - decimal)
TotalPctDiv	Household	percentage of population who are divorced (numeric - decimal)
PctFam2Par	Household	percentage of families (with kids) that are headed by two parents (numeric - decimal)
PctKids2Par	Household	percentage of kids in family housing with two parents (numeric - decimal)
PctYoungKids2Par	Household	percent of kids 4 and under in two parent households (numeric - decimal)
PctTeen2Par	Household	percent of kids age 12-17 in two parent households (numeric - decimal)
PctIlleg	Household	percentage of kids born to never married (numeric - decimal)
PctPersOwnOccup	Household	percent of people in owner occupied households (numeric - decimal)
PctHousOwnOcc	Household	percent of households owner occupied (numeric - decimal)
PctHousNoPhone	Household	percent of occupied housing units without phone (in 1990, this was rare!) (numeric - decimal)

Appendix B – Supporting Visualisations

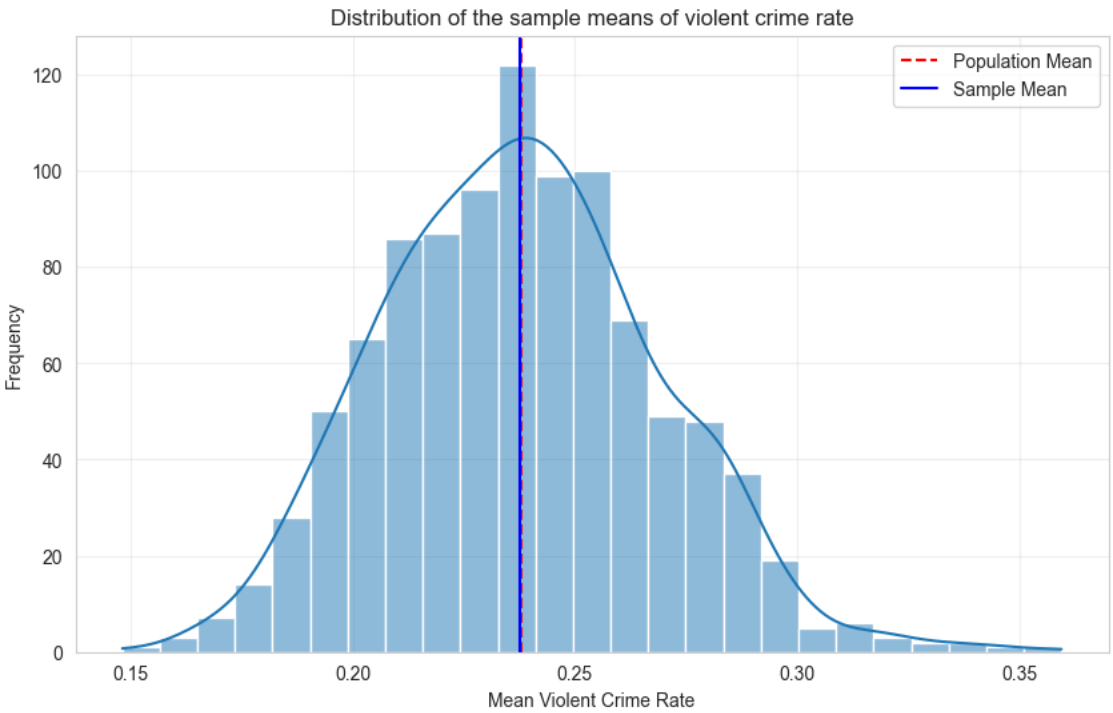
10 Features with Strongest Positive Correlation to Violent Crime: Pearson's r vs. Spearman's p

Feature	**Pearson's *r***	**Spearman's *p***
PctIlleg	0.74	0.74
racepctblack	0.63	0.60
pctWPubAsst	0.57	0.63
FemalePctDiv	0.56	0.64
TotalPctDiv	0.55	0.63
MalePctDivorce	0.53	0.59
PctPopUnderPov	0.52	0.62
PctUnemployed	0.50	0.55
PctHousNoPhone	0.49	0.59
PctNotHSGrad	0.48	0.53

Top 20 Variance Inflation Factors



Distribution of the Sample Means of the Violent Crime Rate



Appendix C – Full Grid Search Implementation

```
best_params = {}
best_history = None
min_error = float('inf')

results_list = []
all_histories = []  # Store all histories for plotting


for lr in [0.0001, 0.01, 1, 1.5]:
    # Include batch gradient descent (full dataset) and mini-batch sizes
    batch_sizes = [32, 64, len(X_train)]

    for bs in batch_sizes:

        np.random.seed(40466218)
        theta_init = np.random.randn(X_train.shape[1])

        best_theta, history = train_linear_regression(
            X_train, y_train, X_val, y_val, theta_init, lr, 2000, bs,
        )

        bs_display = 'Full' if bs == len(X_train) else bs
        print(f"{lr:<8} | {bs_display:<12} | {min(history['val_mse']):<.5f} | {len(history['val_mse']):<10} |")

        results_list.append({
            'Learning Rate': lr,
            'Batch Size': bs_display,
            'Val MSE': min(history['val_mse']),
            'Epochs Run': len(history['val_mse']),
        })

        all_histories.append({
            'learning_rate': lr,
            'batch_size': bs_display,
            'history': history,
        })

        best_val_mse = min(history['val_mse'])
        epochs_run = len(history['val_mse'])

        if best_val_mse < min_error:
            min_error = best_val_mse
            best_params = {'lr': lr, 'batch_size': bs}
            best_history = history
```

Appendix D – Code

```
# %% [markdown]

# # Predict the US Violent Crime Rate in the '90s


# %% [markdown]

# ## Setup Notes

#

# The two requirements to run ths notebook are:

# - Have requirements.txt and variables_enhanced.csv in the folder this is opened in (both contained in the submission)

# - Use pip install requirements.txt to install all necessary libraries.


# %% [markdown]

# ## Data Preparation

#

# ### Data Loading and Inspection

#

# Necessary libraries and data were imported, features (X), target (y), and metadata were parsed, and initial data previews were carried out.


# %%

# Import libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from ucimlrepo import fetch_ucirepo

import scipy.stats as st
```

```
from mdptoolbox import mdp

# Fetch dataset
communities_and_crime = fetch_ucirepo(id=183)

# Set default visualisation style
sns.set_style('whitegrid')
sns.set_palette('tab10')
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3
plt.rcParams['figure.figsize'] = (10, 6)


# %%
# Save data as pandas dataframes
X = communities_and_crime.data.features
y = communities_and_crime.data.targets

# Preview the features
# Note that missing values are represented as ?
print('Table 1:\n')
X

# %%
# Preview the target variable
print('Table 2:\n')
y
```

```
# %%
# Get the variable info from the metadata
metadata = communities_and_crime.metadata
variable_info = metadata['additional_info']['variable_info']

# Parse variable descriptions into a DataFrame
variable_info_df = pd.DataFrame([
    line.strip().replace('--', '').split(':', 1)
    for line in variable_info.split('\n')
    if '--' in line
], columns=['variable', 'info'])

# Merge with variables dataframe on lowercase names
variables = communities_and_crime.variables
variables = variables.merge(
    variable_info_df,
    left_on=variables['name'].str.strip().str.lower(),
    right_on=variable_info_df['variable'].str.strip().str.lower(),
    how='left'
)

# Select and rename columns
variable_descriptions = variables[['variable', 'type', 'missing_values', 'info']]

print('Table 3:\n')
variable_descriptions

# %%
```

```
# The variables descriptions dataframe was enhanced using Gemini to categorise the data by (likely) source and topic

# They were categorised based on the descriptions in the 'info' variable above to be used later for downstream analysis


variables_descriptions_enhanced = pd.read_csv('variables_enhanced.csv')

print('Table 4:\n')

variables_descriptions_enhanced


# %% [markdown]

# ### Handling Missing Values

#

# '?' markers were replaced with NaN, missing value percentages were analysed across features, and columns exceeding an 80% missing threshold or identified as non-predictive by the data authors were dropped.


# %%

# Set '?' to NaN and analyze missing values

X = X.copy().replace('?', np.nan)


null_df = (X.isnull().sum()

        .sort_values(ascending=False)

        .to_frame(name='missing')

        .query('missing > 0')

        .reset_index())


# Merge with variable info

null_df = null_df.merge(

   variables_descriptions_enhanced,
```

```
    left_on=null_df['index'].str.strip().str.lower(),
    right_on=variables_descriptions_enhanced['variable'].str.strip().str.lower(),
    how='left'
)

# Keep only the relevant columns and add the % missing
null_df = null_df[['variable', 'info', 'source', 'topic', 'missing']]
null_df['missing_pct'] = round(null_df['missing'] / X.shape[0], 2)

print('Table 5:\n')
null_df

# %%
# Identify features with over 80% missing values
over_threshold = null_df['variable'][null_df['missing_pct'] > 0.8].str.strip()

print(f'Variables with over 80% missing values:\n{over_threshold}')

# Remove these features from X as there is not enough data to impute them reliably
print('Table 6:\n')
X_clean = X.drop(columns=over_threshold)


# %%
# Calculate the number and percentage of zeros in the LemasPctOfficDrugUn variable
lemas_drug_un_zeros = (X_clean['LemasPctOfficDrugUn'] == 0).sum()
lemas_drug_un_total = X_clean['LemasPctOfficDrugUn'].size
```

```
print(f'{lemas_drug_un_zeros} of the {lemas_drug_un_total} values for LemasPctOfficDrugUn are zero: ({((lemas_drug_un_zeros / lemas_drug_un_total)*100):.0f}%)')


# %%

# Review all columns identified as 'not predictive' in the variable descriptions

not_predictive_vars = variable_descriptions['info'].str.contains('predictive')

print('Table 7:\n')

variable_descriptions[not_predictive_vars]


# %%

# Drop the non-predictive features with over 50% missing values + LemasPctOfficDrugUn

# We will also drop state and fold later in the process

X_clean = X_clean.copy().drop(columns=['community','county','LemasPctOfficDrugUn'])


# %% [markdown]

# ## Data Transformation

#

# ### Imputing Missing Values

#

# The single missing value in the `OtherPerCap` feature was imputed using the mean value for its state (Mississippi).


# %%

# Look at the null value for Natchezcity in context of others in its state (28 : Mississippi)

otherpercap_state_28 = X_clean[X_clean['state'] == 28][['state', 'communityname', 'OtherPerCap']]
```

```
state_28_mean = otherpercap_state_28['OtherPerCap'].astype(float).mean()

total_mean = X_clean['OtherPerCap'].astype(float).mean()


print('Table 8:\n')

print(otherpercap_state_28)

print(f"\nThe mean OtherPerCap for Mississippi (state 28) is {state_28_mean.round(2)}")

print(f"\nThe mean OtherPerCap for the entire dataset is {total_mean.round(2)}")


# %%

# Replace the null value in row 13 with the mean of the state

X_clean_and_complete = X_clean.copy()

X_clean_and_complete.loc[130, 'OtherPerCap'] = state_28_mean


# Check the result

print('Table 9:\n')

print(f'Original Data:\n {X_clean.loc[130][["communityname", "OtherPerCap"]]}')

print(f'\nAfter Manual Imputation:\n
{X_clean_and_complete.loc[130][["communityname", "OtherPerCap"]]}')


# %%

# Review the columns with the 'object' type before and after cleaning

string_data = X.select_dtypes(include=['object'])

print(f'Original "object" columns in X:\n {string_data.columns}\n')


new_string_data = X_clean_and_complete.select_dtypes(include=['object'])

print(f'New "object" columns in X_clean_and_complete:\n {new_string_data.columns}')


# Note most of the 'object' type columns were misclassified numeric columns due to
the presence of '?' representing missing values
```

```
# Now we just need to convert 'OtherPerCap' to numeric after the data imputation


# %%

# Perform finishing touches: converting OtherPerCap to float and dropping final non-predictive features

X_final = X_clean_and_complete.copy()

X_final['OtherPerCap'] = X_final['OtherPerCap'].astype(float)

X_final = X_final.drop(columns=['state', 'fold'])


# Show the one remaining 'object' column - the one true string/categorical variable

print('Table 10:\n')

X_final.select_dtypes(include=['object'])


# %% [markdown]

# ### Handling Categorical Variables

#

# `communityname` was identified as the sole categorical feature and one-hot encoding (OHC) was applied to it. It is identified as 'non-predictive' by the data authors, so we will conduct analysis with and without the OHC data at the final stage.


# %%

# Inspect the variables identified as 'categorical' in the variables metadata

print('Table 11:\n')

variables[variables['type'] == 'Categorical']


# %%

# Import OneHotEncoder from sklearn and apply it to the communityname feature

from sklearn.preprocessing import OneHotEncoder
```

```
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)


categorical_data = X_final['communityname'].to_numpy().reshape(-1, 1) # The encoder expects a 2d array

one_hot_encoded_data = ohe.fit_transform(categorical_data)

original_data = X_final['communityname']


# Show example of one hot encoding result

print('Table 12:\n')

print(f'One Hot Encoded Data:\n{one_hot_encoded_data}\n')

print(f'Data shape:\n{one_hot_encoded_data.shape}: {one_hot_encoded_data.shape[0]} rows and {one_hot_encoded_data.shape[1]} newly created columns.\n')

print(f'One Hot Encoder Feature Names:\n{ohe.get_feature_names_out()}\n')

print(f'Original Categorical Variable:\n{original_data.sort_values()}')


# We can see the 3rd row in the data had the value 'Aberdeentown' which translates to a 1 in the second column of the one hot encoded data

# It is the 2nd column because there were two 'Aberdeencity' entries in the original data which were made unique by the OneHotEncoder


# %% [markdown]

# ### Scaling the Data

#

# As per the authors:

# > All numeric data was normalized into the decimal range  0.00-1.00 using an Unsupervised, equal-interval binning method

#

# This feature scaling (normalisation) already applied in the data was displayed visually. Three final datasets - `X_base` (original numeric data), `X_centred` (standardized
```

numeric data), and `X_centred_and_ohc` (standardized numeric data + OHE categorical data) - were created to explore the impact of the preprocessing methods.

```
# %%
# Illustrate the scaling applied to the data: all values between 0 and 1, but not centred around the mean

examples = ['population', 'householdsize', 'racepctblack']

fig, ax = plt.subplots(1, 3, figsize=(15, 5))
for i, ex in enumerate(examples):
    sns.histplot(X_final[ex], bins=20, ax=ax[i])
    ax[i].set_title(f'Distribution of {ex}')
plt.tight_layout()
print('Figure 1:\n')

# %%
# Create 3 distinct feature datasets to evaluate the impact of centreing the data and using OHE data

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

categorical_features = ['communityname']
numeric_features = list(X_final.drop(columns=categorical_features))

# Use full pipeline with one hot encoder and scaling
full_pipeline = ColumnTransformer(
```

```
    transformers=[
        ('num', Pipeline([
            ('scaler', StandardScaler(with_mean=True, with_std=False))
        ]), numeric_features),
        ('cat', Pipeline([
            ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
        ]), categorical_features)
    ]
)

# Dataset #1
X_centred_and_ohe = full_pipeline.fit_transform(X_final)

# Drop categorical variable as it is 'non-predictive' and create centred and uncentred
versions of the numeric data for comparisons

# Dataset #2
scaler = StandardScaler(with_mean=True, with_std=False)
X_centred = scaler.fit_transform(X_final[numeric_features])

# Dataset #3
X_base = X_final[numeric_features].to_numpy()

# %%
# Review the first 5 rows for each dataset
print('Table 13:\n')
print('First 5 rows and columns in each dataset:\n')
print(f'X_centred_and_ohe (full pipeline) first 5:\n {X_centred_and_ohe[:5, :5]}\n')
```

```
print(f'X_centred (numeric centred) first 5:\n {X_centred[:5, :5]}\n')
print(f'X_base (numeric uncentred) first 5:\n {X_base[:5, :5]}\n')

# One-hot encoding
print(f'Full pipeline last 5 (one-hot-encoding):\n {X_centred_and_ohe[5:, -5:]}\n')
```

%% [markdown]

Matrix Decomposition Analysis and Results

#

Singular Value Decomposition (SVD)

#

SVD was applied to the three prepared datasets (`X_base`, `X_centred`, `X_centred_and_ohc`) to decompose them. Principal components were selected from Vh and used to transform the source data.

```
# %%
# Perform SVD on the data and capture the U, s and Vh matrices
def perform_svd(A):
    U, s, Vh = np.linalg.svd(A) # perform SVD on Matrix A

    # Save the rows and columns as m & n
    n = A.shape[0]
    m = A.shape[1]

    # Create m x n Sigma matrix
    Sigma = np.zeros((m, n))
    np.fill_diagonal(Sigma, s)
```

```
    return U, s, Vh, Sigma

U, s, Vh, Sigma = perform_svd(X_centred_and_ohe)

# Display a preview and capture the structure of each of the matrices
print('Table 14:\n')
print(X_centred_and_ohe)
print(f'\nM Matrix = m x n = {X_centred_and_ohe.shape}\n')
print(U)
print(f'\nU Matrix = m x m = {U.shape}\n')
print(s)
print(f'\ns Matrix = n 1-d array = {s.shape}\n')
print(Sigma)
print(f'\nDiagonalised s Matrix = m x n = {Sigma.shape}\n')
print(Vh)
print(f'\nvH Matrix = n x n = {Vh.shape}\n')


# %%
# Create a function to select a subset of principal components from Vh and transform the original data using their dot product

def reduce_dimensions(df, components):
    U, s, Vh = np.linalg.svd(df)
    Vh_reduced = Vh[:components, :]
    X_svd = df.dot(Vh_reduced.T)

    return X_svd
```

```
# Capture the SVD outputs for the three datasets

X_centred_and_ohe_svd = reduce_dimensions(df=X_centred_and_ohe, components=5)

# Preview an example

print('Table 15:\n')
print(f'Displaying the original and transformed matrices after SVD has been applied and 5 selected:\n')
print('First 5 rows and columns in each dataset:\n')
print(f'Original:\n {X_centred_and_ohe[:5,:5]}\n')
print(f'Transformed:\n {X_centred_and_ohe_svd[:5,:5]}\n')

# %% [markdown]
# ### Principal Component Analysis (PCA)
#
# The explained variance ratios were compared and visualised for the each of the three datasets. Centreing the data was demonstrated to be a prerequisite for SVD and PCA and the base data was dropped from future analysis. Cumulative explained variance was plotted on an elbow plot for the two remaining datasets (centred, with or without the OHC variables) to identify the ideal number of principal components to explain sufficient variance while sufficiently reducing the dimensionality in the data. Principal components loadings were inspected and visualised to indicate what the principal components represent.
#
# 25 principal components explained over 90% of the variance while reducing the dimensonality by a factor of 4, while 50 components explained over 95%. Both were chosen as options to move forward with the final analysis.

# %%
```

```
# Create a function to calculate the explained variance ratio by squaring the singular values in s to get the variance and then dividing by the total variance


def calculate_explained_variance(df):
    U, s, Vh = np.linalg.svd(df)
    explained_variance_ratio = (s ** 2) / np.sum(s ** 2)


    return explained_variance_ratio


# Capture the explained variance ratio for the three datasets


X_centred_and_ohe_variance = calculate_explained_variance(df=X_centred_and_ohe)
X_centred_variance = calculate_explained_variance(df=X_centred)
X_base_variance = calculate_explained_variance(df=X_base)
# Preview an example
X_centred_and_ohe_variance


# %%
# Gather the explained variance ratio arrays into list and then a dataframe for a quick comparison of the principal components values
variance_explained = [X_centred_and_ohe_variance, X_centred_variance, X_base_variance]


variance_explained_comparison = pd.DataFrame(variance_explained)
variance_explained_comparison = variance_explained_comparison.T.rename({0:'Centred + OHC', 1:'Centred', 2:'Base'}, axis=1)


print('Table 16:\n')
```

```
variance_explained_comparison.head(10)



# %%

# Display the variance explained ratio for each of the three datasets on the same scale to highlight the outsized impact of PC1 for the base dataset


num_components = 10


titles = ['Centred + OHE', 'Centred', 'Base']


# Trying to plot in a 3x1 subplot made the code functional but barely comprehensible: stacking 3 charts is more feasible

for i, v in enumerate(variance_explained):

    # Visualise explained variance ratios

    plt.bar(range(1, num_components + 1), v[:num_components], align="center")

    plt.xlabel("Principal Component")

    plt.ylabel("Explained Variance Ratio")

    plt.title(f'{titles[i]} Dataset: Explained Variance Ratio per PC')

    plt.tight_layout()

    plt.ylim(0, .85) # display them on the same scales to highlight the outsized impact of PC1 for the base dataset

    plt.show()

print('Figure 3:\n')


# %%

# Calculate the cumulative explained variance for the two remaining datasets: centred and centred with one hot encoding
```

```
cum_exp_var_with_ohe = np.cumsum(X_centred_and_ohe_variance)

cum_exp_var_without_ohe = np.cumsum(X_centred_variance)

cum_exp_var_list = [cum_exp_var_with_ohe, cum_exp_var_without_ohe]


components_displayed = 70


# Plot an elbow plot for both with visual thresholds for 80%, 90% and 95% explained
variance

titles = ['Centred + OHE', 'Centred']

for i, data in enumerate(cum_exp_var_list):

    plt.plot(range(1, components_displayed + 1), data[:components_displayed],
marker='o', alpha=0.4)

    plt.axhline(y=0.8, color='grey', linestyle='--', label='80% Explained Variance')

    plt.axhline(y=0.9, color='red', linestyle='--', label='90% Explained Variance')

    plt.axhline(y=0.95, color='green', linestyle='--', label='95% Explained Variance')

    plt.axvline(x=25, color='blue', linestyle='-', label='25 Components')

    plt.title(f"{titles[i]} Dataset: Cumulative Explained Variance (SVD)")

    plt.ylabel("Cumulative Explained Variance")

    plt.xlabel("Number of Principal Components")

    plt.tight_layout()

    plt.legend()

    plt.show()

print('Figure 4:\n')



# %%

# Select the final datasets for analysis
```

```
X_centred_and_ohe_svd_25 = reduce_dimensions(df=X_centred_and_ohe, components=25)

X_centred_and_ohe_svd_50 = reduce_dimensions(df=X_centred_and_ohe, components=50)

X_centred_svd_25 = reduce_dimensions(df=X_centred, components=25)

X_centred_svd_50 = reduce_dimensions(df=X_centred, components=50)


# %%

# Calculate the principal components loadings, align them to the original data labels and review


def calculate_loadings(data=X_centred, principal_component = 0):


    U, s, Vh = np.linalg.svd(data)


    # Get loadings for PC (row of Vh)

    pc_loadings = Vh[principal_component, :]

    # Convert to a series, add labels, sort and get the top 5

    loadings_series = pd.Series(pc_loadings, index=numeric_features)

    sorted_loadings = loadings_series.abs().sort_values(ascending=False)

    top_5_features = sorted_loadings.head(5)


    print(f"Top 5 loadings for PC{principal_component + 1}:")

    print(f'{loadings_series.loc[top_5_features.index]}\n')


print('Table 17:\n')

for i in range(0,3):

    calculate_loadings(principal_component = i)
```

```
# %%
# PC1 and Income
plt.scatter(X_centred_svd_25[:, 0], X_final['medFamInc'], alpha=0.5)
plt.xlabel('Principal Component 1')
plt.ylabel('Median Family Income')
plt.title('PC1 is strongly correlated with income');
print('Figure 5:\n')

# %%
# PC2 and Immigration
plt.scatter(X_centred_svd_25[:, 1], X_final['PctRecImmig10'], alpha=0.5)
plt.xlabel('Principal Component 2')
plt.ylabel('% of immigrants who have immigrated in last 10 years')
plt.title('PC2 is strongly correlated with immigration');
print('Figure 6:\n')

# %%
# PC3 and Household Size
plt.scatter(X_centred_svd_25[:, 2], X_final['PersPerOccupHous'], alpha=0.5)
plt.xlabel('Principal Component 3')
plt.ylabel('Mean persons per household')
plt.title('PC3 is strongly correlated with household size');
print('Figure 7:\n')

# %%
# Counter-example: no relation between PC3 and immigration demonstrates that
principal components are representing distinct, independent features
plt.scatter(X_centred_svd_25[:, 2], X_final['PctRecImmig10'], alpha=0.5, c='grey')
```

```
plt.xlabel('Principal Component 3')

plt.ylabel('% of immigrants who have immigrated in last 10 years')

plt.title('PC3 has no correlation with immigration');

print('Figure 8:\n')


# %%

# Create a pairplot the first 5 principal components


X_centred_svd_5 = reduce_dimensions(df=X_centred, components=5)

X_centred_svd_5_df = pd.DataFrame(X_centred_svd_5, columns=[f'PC{i+1}' for i in
range(X_centred_svd_5.shape[1])])


sns.pairplot(X_centred_svd_5_df);




# %% [markdown]

# ## Linear Regression Results

#

# A linear regression model was fit for each of the two datasets and the two sets of
principal components. They were compared against the original data on RMSE.


# %%

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import root_mean_squared_error


final_datasets = [X_centred_and_ohe_svd_25, X_centred_and_ohe_svd_50,
X_centred_svd_25, X_centred_svd_50]
```

```
def linear_regression_pipeline(data):
    # Split data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(data, y, test_size=0.2,
random_state=40466218) # random_state = my student ID


    # Train a linear regression model using OLS via SVD/pseudoinverse and calculate the
RMSE
    # Add intercept
    X_train_aug = np.hstack([np.ones((X_train.shape[0], 1)), X_train])
    X_test_aug = np.hstack([np.ones((X_test.shape[0], 1)), X_test])
    # Compute pseudoinverse using SVD and coefficients
    beta = np.linalg.pinv(X_train_aug) @ y_train
    # Predict
    y_pred = X_test_aug @ beta
    rmse = root_mean_squared_error(y_test, y_pred)


    return rmse


# Compute RMSE for each dataset
rmse_results = {
    "Original Data":
linear_regression_pipeline(X_final.drop(columns=['communityname'])),  # excludes
columns with missing data and categorical variable
    "Centred + OHE, 25 PCs": linear_regression_pipeline(X_centred_and_ohe_svd_25),
    "Centred + OHE, 50 PCs": linear_regression_pipeline(X_centred_and_ohe_svd_50),
    "Centred, 25 PCs": linear_regression_pipeline(X_centred_svd_25),
    "Centred, 50 PCs": linear_regression_pipeline(X_centred_svd_50)
}
```

```
print('Table 18:\n')

# Convert to a DataFrame and display

rmse_results_df = pd.DataFrame((rmse_results.items()), columns=["Dataset", "RMSE"])

rmse_results_df



# %%

# Display data shapes for final reference to dataset dimensionality

print(f"Original Data Shape: {X_final.drop(columns=['communityname']).shape}")

print(f"50 Principal Components: {X_centred_svd_50.shape}")

print(f"25 Principal Components: {X_centred_svd_25.shape}")


# %% [markdown]

# In conclusion, applying dimensionality reduction weakened the predictive performance of the model: a simple linear regression model on the original data performed best. For this dataset it improves model efficiency by reducing the size of the data significantly, but it does not improve model performance by reducing noise.

#

# This is a typical outcome and it requires a judgment call about whether there is a need for a more efficient model (reduced costs, faster inference). If so, PCA using SVD effectively retains a significant amount of the signal in the data while making it more efficient.

#

# The inclusion of the one hot encoded categorical variable made a negligible impact while substantially increasing the dataset size: it requires no judgement call, it should be excluded from analysis as per the data authors' recommendation.


# %% [markdown]

# ## Exploratory Data Analysis
```

```
#

# ### Distributions

#

# The right-skewed nature of the target variable is illustrated through the histogram and summary statistics.

#

# While most communities have a very low violent crime rate (median = 0.15), 44 communities have a very high violent crime rate (1, more than 6x larger than the typical community). The communities with the highest violent crime rates are disproportionately represented in cities.


# %%

# Display a histogram of the target variable


sns.histplot(y, kde=True)

plt.title('Histogram of Violent Crime Rate')

plt.xlabel('Violent Crime Rate')

plt.ylabel('Frequency');


# %%

# Display summary statistics of the target variable


y.describe()


# %%

# Measure the skewness of the target variable

y.skew()


# Measure the kurtosis of the target variable
```

```
y.kurtosis()

# Print the skewness and kurtosis of the target variable
print(f"Skewness: {y.skew()}")
print(f"Kurtosis: {y.kurtosis()}")


# %%
# Combine features and target variable

full_dataset = pd.concat([X_final, y], axis=1)

# %% [markdown]
# ### Correlations
#
# The 10 features with the strongest correlations (positive and negative) with the target variable are displayed in a correlation table and heatmap. A sample of features representing each category - crime, economy, household makeup and race - are shown in a pairplot to display the relationships between them.

# %%
# Calculate the two correlation coefficients between violent crime rate and each of the key features

spearman_correlations = full_dataset.corr(method='spearman', numeric_only=True)['ViolentCrimesPerPop']
pearson_correlations = full_dataset.corr(method='pearson', numeric_only=True)['ViolentCrimesPerPop']

comparison_df = pd.DataFrame({
```

```
    'Feature': pearson_correlations.index,
    'Pearson Correlation': pearson_correlations.values,
    'Spearman Correlation': spearman_correlations.values,
    'Absolute Difference': pd.Series(spearman_correlations.values - pearson_correlations.values).abs().round(2)
})

comparison_df.sort_values(by='Pearson Correlation', ascending=False).reset_index(drop=True).head(11)

# %%
# Plot nonlinear relationship between violent crime rate and poverty rate
# Plot curve of best fit, add Pearson's r and Spearman's ρ in the top right

orders = [1, 2, 5]

# Take the Pearson's r and Spearman's rho for PctPopUnderPov from comparison_df above
pearson_r = comparison_df.loc[comparison_df['Feature'] == 'PctPopUnderPov', 'Pearson Correlation'].iloc[0]
spearman_rho = comparison_df.loc[comparison_df['Feature'] == 'PctPopUnderPov', 'Spearman Correlation'].iloc[0]

fig, axes = plt.subplots(1, 3, figsize=(18, 6), sharex=True)
for i, order in enumerate(orders):
    ax = axes[i]
    # Plot regression line only
    sns.regplot(
        data=full_dataset,
```

```
    x='PctPopUnderPov',
    y='ViolentCrimesPerPop',
    scatter=False,
    order=order,
    ax=ax,
    color='C0',
    label=f'Order {order}'
)
# Plot all data as scatter in grey
ax.scatter(
    full_dataset['PctPopUnderPov'],
    full_dataset['ViolentCrimesPerPop'],
    color='grey', alpha=0.3, s=10, label="Data"
)
ax.set_xlabel('Poverty Rate')
ax.set_ylabel('Violent Crime Rate')
ax.set_title(f'Nonlinear Relationship (Order {order})')
# In the middle plot, add Pearson and Spearman in top right
if i == 1:
    stats_text = (
        f"Pearson's r = {pearson_r:.2f}\n"
        f"Spearman's ρ = {spearman_rho:.2f}"
    )
    # Place text in top right with a little padding
    ax.text(
        0.98, 0.98, stats_text,
        transform=ax.transAxes,
        ha='right', va='top', fontsize=11,
```

```
        bbox=dict(facecolor='white', alpha=0.7, edgecolor='none')
    )
  ax.legend()
plt.tight_layout();

# %%
def create_bins(df, variable, bin_col_name, n_bins=10):
  bins = np.linspace(df[variable].min(), df[variable].max(), n_bins + 1)
  bin_labels = [f'{bins[i]:.1f} - {bins[i+1]:.1f}' for i in range(len(bins) - 1)]
  df[bin_col_name] = pd.cut(df[variable], bins=bins, labels=bin_labels,
include_lowest=True)
  return df

# Create bings for poverty, race and crime for later use
full_dataset = create_bins(full_dataset, 'PctPopUnderPov', 'PovBin10', n_bins=10)
full_dataset = create_bins(full_dataset, 'PctPopUnderPov', 'PovBin5', n_bins=5)
full_dataset = create_bins(full_dataset, 'PctPopUnderPov', 'PovBin3', n_bins=3)
full_dataset = create_bins(full_dataset, 'racePctWhite', 'RaceWhiteBin3', n_bins=3)
full_dataset = create_bins(full_dataset, 'racePctWhite', 'RaceWhiteBin5', n_bins=5)
full_dataset = create_bins(full_dataset, 'racepctblack', 'RaceBlackBin3', n_bins=3)
full_dataset = create_bins(full_dataset, 'racepctblack', 'RaceBlackBin5', n_bins=5)
full_dataset = create_bins(full_dataset, 'ViolentCrimesPerPop', 'CrimeBin3', n_bins=3)
full_dataset = create_bins(full_dataset, 'ViolentCrimesPerPop', 'CrimeBin5', n_bins=5)

# %%
# Show how the distribution of violent crime rates varies across community types
sns.boxplot(data = full_dataset, x = 'PovBin10', y = 'ViolentCrimesPerPop')
plt.title('Violent Crime Rate by 10-Binned Poverty Rate')
```

```
plt.xlabel('Poverty Rate - 10 Bins')

plt.ylabel('Violent Crime Rate');


# %%

# Show the counts of communities in each 10-binned poverty rate


full_dataset['PovBin10'].value_counts(sort=False)


# %%

# Calculate the correlations between the features and the target variable, sort them and display the top and bottom 10

top_10_features_correlations = full_dataset.corr(method='pearson', numeric_only=True).sort_values(by='ViolentCrimesPerPop', ascending=False).head(11)['ViolentCrimesPerPop'].iloc[1:]

bottom_10_features_correlations = full_dataset.corr(method='pearson', numeric_only=True).sort_values(by='ViolentCrimesPerPop', ascending=False).tail(10)['ViolentCrimesPerPop']


print(f"10 features most correlated with violent crime rate:\n{top_10_features_correlations}\n")

print(f"10 features least correlated with violent crime rate:\n{bottom_10_features_correlations}\n")



# %%

# Save descriptions of the top/bottom 10 features to a csv file


combined_top_bottom_10_features_correlations = pd.concat([top_10_features_correlations, bottom_10_features_correlations], axis=1)

features_list = combined_top_bottom_10_features_correlations.index
```

```
variables_descriptions_enhanced[variables_descriptions_enhanced['variable'].isin(features_list)][['variable', 'info']].to_csv('top_bottom_10_features_descriptions.csv', index=False)
```

```
# %%
# For the feature categories with multiple variables - income, divorce, household ownership, race and family size - I selected the features with strongest correlations to keep the feature set manageable
# And to observe correlations between distinct categories, rather than variables within categories (which are high by default)
key_features = ['ViolentCrimesPerPop', 'pctWPubAsst', 'FemalePctDiv', 'PctPopUnderPov', 'PctUnemployed', 'PctHousNoPhone', 'PctNotHSGrad', 'PctKids2Par', 'pctWInvInc', 'PctPersOwnOccup', 'medIncome', 'racePctWhite']
key_features_df = full_dataset[key_features]

key_features_df.corr(numeric_only=True)
```

```
# %%
# Visualise as a heatmap

sns.heatmap(key_features_df.corr(method='pearson', numeric_only=True), cmap='coolwarm')
```

```
# %%
# Selected a key feature from distinct categories - economy, education, household makeup and race - to display a pairplot of the relationships between them

distinct_feature_categories = ['ViolentCrimesPerPop', 'PctPopUnderPov', 'PctNotHSGrad', 'PctKids2Par', 'racePctWhite']
```

```
distinct_feature_categories_df = full_dataset[distinct_feature_categories]

sns.pairplot(distinct_feature_categories_df);



# %%
race_black_bins = create_bins(full_dataset, 'racepctblack', 'RaceBlackBin3', n_bins=3)

sns.pointplot(
    data=full_dataset,
    x='PovBin5', # 5-binned poverty rate
    y='ViolentCrimesPerPop',
    hue='RaceBlackBin3', # 3-binned black population rate
    capsize=0.1,
    dodge=True,
)

plt.title('Interaction Effect: Poverty and Race vs. Crime')
plt.ylabel('Average Violent Crime Rate')
plt.xlabel('Poverty Level')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# %%
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

X_vif_data = X_final.select_dtypes(include=[np.number])
```

```
X_vif_with_const = add_constant(X_vif_data)

vif_values = []
for i in range(X_vif_with_const.shape[1]):
    try:
        # calculate VIF for each feature
        vif_val = variance_inflation_factor(X_vif_with_const.values, i)
    except Exception:
        vif_val = np.nan
    vif_values.append(vif_val)

vif = pd.Series(vif_values, index=X_vif_with_const.columns)

if 'const' in vif.index:
    vif = vif.drop('const')

top_vif = vif.sort_values(ascending=False).head(20)
plt.figure(figsize=(10, 6))
sns.barplot(x=top_vif.values, y=top_vif.index, color='skyblue')
plt.axvline(10, color='red', linestyle='--', label='Threshold: 10')
plt.xlabel('VIF')
plt.ylabel('Features')
plt.title('Top 20 Variance Inflation Factors')
plt.legend()
plt.tight_layout();

# %% [markdown]
# ### Hypotheses
```

```
#

# 4 hypotheses are tested:

#

# - H<sub>1</sub> - Communities with a high poverty rate have significantly different crime rates to communities with low poverty rates.

# - H<sub>2</sub> - Communities with a high % of white people have significantly different crime rates to communities with low % of white people.

#

# The significance level for the tests is set to p < 0.05.

#

# Null hypotheses for H<sub>1</sub> and H<sub>2</sub> are rejected.


# %%

# We know that the target variable is right-skewed which violates the normality assumption for the t-test


sns.histplot(full_dataset['ViolentCrimesPerPop'], kde=True)

plt.title('Distribution of Violent Crime Rate');

plt.xlabel('Violent Crime Rate')

plt.ylabel('Frequency')


# %%

# But the central limit theorem shows that the sample mean is approximately normally distributed at this sample size


# Display the central limit theorem in action using the mean violent crime rate

population = full_dataset['ViolentCrimesPerPop']

n_simulations = 1000

sample_size = 50
```

```
sample_means = []

# Sample the data, capture the mean and iterate 1000 times
for _ in range(n_simulations):
    sample = population.sample(sample_size)
    sample_means.append(sample.mean())

# Display the distribution of the sample means
sns.histplot(sample_means, kde=True)
plt.axvline(population.mean(), color='red', linestyle='--', label='Population Mean')
plt.axvline(np.mean(sample_means), color='blue', linestyle='-', label='Sample Mean')
plt.title('Distribution of the sample means of violent crime rate')
plt.xlabel('Mean Violent Crime Rate')
plt.ylabel('Frequency')
plt.legend();

# Display the first 10 sample means
print('The first 10 sample means are:')
print(sample_means[:10])

# %%
median_poverty_rate = full_dataset['PctPopUnderPov'].median()

# Split the dataset into high and low poverty communities
high_poverty_rate_communities = full_dataset[full_dataset['PctPopUnderPov'] >
median_poverty_rate]
```

```
low_poverty_rate_communities = full_dataset[full_dataset['PctPopUnderPov'] <=
median_poverty_rate]


poverty_rate_results =
st.ttest_ind(high_poverty_rate_communities['ViolentCrimesPerPop'],
low_poverty_rate_communities['ViolentCrimesPerPop'], equal_var=False)


# Display the mean violent crime rate for the two groups

print(f"High poverty communities' violent crime rate:
Mean:{high_poverty_rate_communities['ViolentCrimesPerPop'].mean():.2f},
Variance:{high_poverty_rate_communities['ViolentCrimesPerPop'].var():.2f}")

print(f"Low poverty communities' violent crime rate:
Mean:{low_poverty_rate_communities['ViolentCrimesPerPop'].mean():.2f},
Variance:{low_poverty_rate_communities['ViolentCrimesPerPop'].var():.2f}")


if poverty_rate_results.pvalue < 0.05:

    print('\n✔ Hypothesis 1: Statistically Significant')

    print("\nThe mean violent crime rate for high poverty communities is significantly
different to the mean violent crime rate for low poverty communities.\n")

    print(f'p-value: {poverty_rate_results.pvalue}')

    print(f't-value: {poverty_rate_results.statistic: .2f}')

    print(f'degrees of freedom: {poverty_rate_results.df: .0f}')

else:

    print('\n✗ Hypothesis 1: Not Statistically Significant')

    print("\nThe mean violent crime rate for high poverty communities is not significantly
different to the mean violent crime rate for low poverty communities.")

    print(f'p-value: {poverty_rate_results.pvalue}')


# %%

median_race_pct_white = full_dataset['racePctWhite'].median()
```

```
# Split the dataset into high and low % of white communities

high_race_pct_white_communities = full_dataset[full_dataset['racePctWhite'] >
median_race_pct_white]

low_race_pct_white_communities = full_dataset[full_dataset['racePctWhite'] <=
median_race_pct_white]


race_pct_white_results =
st.ttest_ind(high_race_pct_white_communities['ViolentCrimesPerPop'],
low_race_pct_white_communities['ViolentCrimesPerPop'], equal_var=False)


# Display the mean violent crime rate for the two groups

print(f"High % of white communities' violent crime rate:
{high_race_pct_white_communities['ViolentCrimesPerPop'].mean():.2f},
Variance:{high_race_pct_white_communities['ViolentCrimesPerPop'].var():.2f}")

print(f"Low % of white communities' violent crime rate:
{low_race_pct_white_communities['ViolentCrimesPerPop'].mean():.2f},
Variance:{low_race_pct_white_communities['ViolentCrimesPerPop'].var():.2f}")



if race_pct_white_results.pvalue < 0.05:

   print('\n✔ Hypothesis 2: Statistically Significant')

   print("\nThe mean violent crime rate for high % of white communities is significantly
different to the mean violent crime rate for low % of white communities.\n")

   print(f'p-value: {race_pct_white_results.pvalue}')

   print(f't-value: {race_pct_white_results.statistic: .2f}')

   print(f'degrees of freedom: {race_pct_white_results.df: .0f}')

else:

   print('\n✗ Hypothesis 2: Not Statistically Significant')

   print("\nThe mean violent crime rate for high % of white communities is not
significantly different to the mean violent crime rate for low % of white communities.")

   print(f'p-value: {race_pct_white_results.pvalue}')
```

```
# %% [markdown]
# ## Model Optimisation
#
# ### Model Training

# %%
# Helper functions for linear regression
def add_bias(X):
    return np.hstack([np.ones((len(X), 1)), X])

def lin_reg_predict(X_b, theta):
    y_pred = X_b.dot(theta)
    return y_pred

def lin_mse(y, y_pred):
    return np.mean((y - y_pred) ** 2)

def lin_reg_gradient(X_b, y, alpha, theta):
    m = len(y)

    # Base gradient
    gradients = (2/m) * X_b.T.dot(X_b.dot(theta) - y)

    # Regularisation logic
    if alpha > 0:
        reg_term = 2 * alpha * theta.copy()
```

```
        reg_term[0] = 0  # Zero out bias penalty
        gradients += reg_term

    return gradients



# Optimal hyperparameter values for momentum, patience, decay and alpha can be
found in Appendix C
def train_linear_regression(X_train, y_train, X_val, y_val, theta, lr, epochs, batch_size,
momentum=0.7, patience=10, decay=0.9, alpha=0.1):
    m = len(y_train)

    velocity = np.zeros_like(theta)

    # Store training history
    history = {'train_mse': [], 'val_mse': [], 'lr': []}
    best_mse = float('inf')
    best_theta = theta.copy()
    no_improve = 0

    for epoch in range(epochs):
        # Shuffle Data for Mini-Batch
        indices = np.random.permutation(m)
        X_shuffled = X_train[indices]
        y_shuffled = y_train[indices]

        # Mini-Batch Loop
        for i in range(0, m, batch_size):
            xi = X_shuffled[i:i+batch_size]
```

```
    yi = y_shuffled[i:i+batch_size]

    gradients = lin_reg_gradient(xi, yi, alpha, theta)

    velocity = (momentum * velocity) - (lr * gradients)

    # Update Parameters
    theta += velocity

# Evaluation (End of Epoch)
# Calculate MSEs
train_pred = lin_reg_predict(X_train, theta)
val_pred = lin_reg_predict(X_val, theta)

train_mse = lin_mse(y_train, train_pred)
val_mse = lin_mse(y_val, val_pred)

history['train_mse'].append(train_mse)
history['val_mse'].append(val_mse)
history['lr'].append(lr)

# Safety check
if lr < 1e-7:
    print("Learning rate is too low. Stopping training.")
    break

# Early Stopping & Decay Logic
```

```
    # If the validation MSE is lower than the best MSE, update the best MSE and best theta
    if val_mse < best_mse:
        best_mse = val_mse
        best_theta = theta.copy()
        no_improve = 0
    else:
        # If the validation MSE is higher than the best MSE, increment the no_improve counter
        no_improve += 1
        # If the no_improve counter is greater than the patience, reduce the learning rate
        if no_improve >= patience:
            lr *= decay
            no_improve = 0
            # Reset velocity on decay to prevent overshooting
            velocity = np.zeros_like(theta)

            if lr < 1e-7:
                print(f"Early stopping triggered at epoch {epoch}")
                break

    return best_theta, history



# %%
# Prepare data and split into train/validation/test sets


# Prepare numpy arrays
X_np = np.asarray(X_centred)
```

```
y_np = np.asarray(y).flatten()

# Add bias term
X_b = add_bias(X_np)

# Split into train (60%), validation (20%), and test (20%) sets
X_temp, X_test, y_temp, y_test = train_test_split(
    X_b, y_np, test_size=0.2, random_state=40466218
)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=40466218  # 0.25 * 0.8 = 0.2 of original
)

print(f"Training set size: {X_train.shape[0]}")
print(f"Validation set size: {X_val.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")

# Initialize parameters with random seed
np.random.seed(40466218)
n_features = X_b.shape[1]
initial_theta = np.random.randn(n_features)

print(f"\nInitial theta (first 5 values): {initial_theta[:5]}")


# %%
# Make predictions using the initial parameters
```

```
y_pred_initial = lin_reg_predict(X_train, initial_theta)

# Calculate the MSE of the initial predictions
mse_initial = np.mean((y_train - y_pred_initial) ** 2)

print(f"Initial MSE: {mse_initial:.4f}")

# %% [markdown]
# ### Hyperparameter Tuning & Optimization Strategy

# %%
# Grid search across learning rates and batch sizes
# Batch size = len(X_train) tests batch gradient descent; smaller values test mini-batch gradient descent
print(f"{'LR':<8} | {'Batch Size':<12} | {'Val MSE':<6} | {'Epochs Run':<10} |")
print("-" * 40)

best_params = {}
best_history = None
min_error = float('inf')

results_list = []
all_histories = []  # Store all histories for plotting


for lr in [0.0001, 0.01, 1, 1.5]:
    # Include batch gradient descent (full dataset) and mini-batch sizes
    batch_sizes = [32, 64, len(X_train)]
```

```
    for bs in batch_sizes:

        np.random.seed(40466218)
        theta_init = np.random.randn(X_train.shape[1])

        best_theta, history = train_linear_regression(
            X_train, y_train, X_val, y_val, theta_init, lr, 2000, bs,
        )

        bs_display = 'Full' if bs == len(X_train) else bs
        print(f"{lr:<8} | {bs_display:<12} | {min(history['val_mse']):<.5f} |
{len(history['val_mse']):<10} |")

        results_list.append({
            'Learning Rate': lr,
            'Batch Size': bs_display,
            'Val MSE': min(history['val_mse']),
            'Epochs Run': len(history['val_mse']),
        })

        all_histories.append({
            'learning_rate': lr,
            'batch_size': bs_display,
            'history': history,
        })

        best_val_mse = min(history['val_mse'])
```

```
    epochs_run = len(history['val_mse'])

    if best_val_mse < min_error:
        min_error = best_val_mse
        best_params = {'lr': lr, 'batch_size': bs}
        best_history = history


print(f"\nBest Params: {best_params}")

# %% [markdown]
# The best results all have an alpha of 0.01. Batch sizes are typically 128, but can be 32, 64 or 128 - never full. Learning rate is most commonly 0.1, but can be 0.05 or 0.001. Momentum is either 0.7 or 1.0. Patience is 1, 10 or 50. Decay is most commonly 1.0, but can be 0.9, 0.7 or 0.5.

# %%
results_df = pd.DataFrame(results_list)
results_df['Val MSE'] = results_df['Val MSE'].round(4)

results_df

# %%
# Create DataFrame for facet grid plotting
history_data = []
for run in all_histories:
    lr = run['learning_rate']
    bs = run['batch_size']
    history = run['history']
```

```
    epochs = range(1, len(history['val_mse']) + 1)
    for epoch, val_mse in zip(epochs, history['val_mse']):
        history_data.append({
            'Epoch': epoch,
            'Validation MSE': val_mse,
            'Learning Rate': lr,
            'Batch Size': bs
        })

history_df = pd.DataFrame(history_data)

g = sns.FacetGrid(
    history_df,
    col='Batch Size',
    row='Learning Rate',
    hue='Batch Size',        # Adds color differentiation per column
    margin_titles=True,
    height=3,
    aspect=1.3,
    sharey='row',            # Keep sharing row Y-axis for comparison
    palette="viridis"        # Professional, colorblind-friendly palette
)

g.map(sns.lineplot, 'Epoch', 'Validation MSE', linewidth=1.5, alpha=0.8)

g.set(yscale='log')
g.set_axis_labels('Epoch', 'Validation MSE (log)')
g.set_titles(col_template="{col_name}", row_template="LR: {row_name}")
```

```
g.fig.subplots_adjust(top=0.9) # Make space at the top
g.fig.suptitle('Training Dynamics: Effect of Batch Size & Learning Rate', fontsize=16)

plt.show()

# %%
fig, ax1 = plt.subplots(figsize=(10, 6))

ax1.set_xlabel('Epoch')
ax1.set_ylabel('Validation MSE', color='tab:blue')
ax1.plot(best_history['val_mse'], color='tab:blue')
ax1.tick_params(axis='y', labelcolor='tab:blue')

ax2 = ax1.twinx()
ax2.set_ylabel('Learning Rate', color='tab:red')
ax2.step(range(len(best_history['lr'])), best_history['lr'], color='tab:red', linestyle='--')
ax2.tick_params(axis='y', labelcolor='tab:red')

plt.title(f"Training Dynamics: LR Decay & Early Stopping\n(Initial LR={best_params['lr']}, Batch={best_params['batch_size']})");

# %% [markdown]
# ### Final Model Training and Evaluation

# %%
best_lr = best_params['lr']
best_batch_size = best_params['batch_size']
```

```
print(f"\nTraining final model with best hyperparameters:")
print(f"  Learning rate: {best_lr}")
print(f"  Batch size: {best_batch_size}")

# Combine training and validation sets for final training
X_train_final = np.vstack([X_train, X_val])
y_train_final = np.concatenate([y_train, y_val])

# Train final model
np.random.seed(40466218)
final_theta_init = np.random.randn(X_train_final.shape[1])

# Train final model on all training data for 1,500 epochs, no validation/early stopping
final_theta, final_history = train_linear_regression(
    X_train_final, y_train_final, X_train_final, y_train_final, final_theta_init,
    best_lr, 2000, best_batch_size
)

# Plot predictions vs true values
def plot_predictions(y_true, y_pred, title="True vs Predicted Values", xlabel='True
Values', ylabel='Predicted Values'):
    y_true = np.asarray(y_true).ravel()
    y_pred = np.asarray(y_pred).ravel()
    plt.scatter(y_true, y_pred, alpha=0.5)
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.title(title)
    min_val = min(np.min(y_true), np.min(y_pred))
```

```
    max_val = max(np.max(y_true), np.max(y_pred))
    plt.plot([min_val, max_val], [min_val, max_val], 'r--', lw=2)
    plt.show()

y_pred_test = lin_reg_predict(X_test, final_theta)
test_mse = lin_mse(y_test, lin_reg_predict(X_test, final_theta))

print(f"\nFinal model parameters (first 5 values): {final_theta[:5]}")
print(f"Final training MSE: {final_history['train_mse'][-1]:.3f}")
print(f"Test MSE: {test_mse:.3f}")

plot_predictions(y_test, y_pred_test, "Test Set: True vs Predicted Values")

# %%
# Calculate the MSE for SVD using the pseudo-inverse

beta = np.linalg.pinv(X_train_final) @ y_train_final
svd_y_pred = X_test @ beta
mse_svd = lin_mse(y_test, svd_y_pred)
print(f"SVD MSE: {mse_svd:.3f}")

# Plot svd predictions vs gradient descent predictions

plot_predictions(y_pred_test, svd_y_pred, "Gradient Descent Predictions vs SVD and
Pseudo-Inverse Predictions",
"Gradient Descent Predictions",
"SVD and Pseudo-Inverse Predictions")
```

```
# Calculate the correlation coefficient of gradient descent and svd predictions

svd_correlation = np.corrcoef(y_pred_test, svd_y_pred)[0, 1]
print(f"SVD and Gradient Descent Predictions Correlation: {svd_correlation:.3f}")

# %%
# Plot the distribution of the residuals

residuals = y_test - y_pred_test

sns.histplot(residuals, kde=True)
plt.xlabel('Residuals')
plt.ylabel('Frequency')
plt.title('Distribution of Residuals on Training Set');

# Display shapiro-wilk test results

normality_test = st.shapiro(residuals)

if normality_test.pvalue < 0.05:
    print('\n✓ Residuals are normally distributed')
else:
    print('\n✗ Residuals are not normally distributed')

print(f"Shapiro-Wilk test results: w={normality_test.statistic:.3f},
p={normality_test.pvalue}")
```

```
# %%
# Model weight analysis

# Drop the communityname column
features = X_final.drop(columns=['communityname']).columns

# Drop the bias term
coefficients = final_theta[1:]

# Create a DataFrame for the weights
weights_df = pd.DataFrame({
    'Feature': features,
    'Weight': coefficients
})

# Sort the DataFrame by weight in descending order
weights_df = weights_df.sort_values(by='Weight', ascending=False)
top_features = weights_df.head(10)
bottom_features = weights_df.tail(10)

# Plot the weights against the features
plt.barh(top_features['Feature'], top_features['Weight'])
plt.barh(bottom_features['Feature'], bottom_features['Weight'])
plt.xlabel('Model Coefficients')
plt.ylabel('Features')
plt.title('Model Coefficients for Top / Bottom 10 Features (Ridge + Gradient Descent)')

# %%
```

```
# Compare to the coefficients to the SVD and pseudo-inverse implementation

beta_svd = np.linalg.pinv(X_train_final) @ y_train_final
beta_svd = beta_svd[1:]

svd_coefficients = pd.DataFrame({
    'Feature': features,
    'Weight': beta_svd
})

svd_coefficients = svd_coefficients.sort_values(by='Weight', ascending=False)

top_svd_features = svd_coefficients.head(10)
bottom_svd_features = svd_coefficients.tail(10)

plt.barh(top_svd_features['Feature'], top_svd_features['Weight'])
plt.barh(bottom_svd_features['Feature'], bottom_svd_features['Weight'])
plt.xlabel('Model Coefficients')
plt.ylabel('Features')
plt.title('Model Coefficients for Top / Bottom 10 Features (SVD)')



# %%
bottom_svd_features

# %%
# Error analysis
```

```
# Fit the full dataset to the model
X_full = np.asarray(X_centred)
y_full = np.asarray(y).flatten()
X_full_b = add_bias(X_full)
y_pred_full = lin_reg_predict(X_full_b, final_theta)

# Calculate the absolute error between the predicted and true values
absolute_error = np.abs(y_full - y_pred_full)

full_dataset['ViolentCrimesPerPop_pred'] = y_pred_full
full_dataset['absolute_error'] = absolute_error


# %%
predictions_below_0 = full_dataset[full_dataset['ViolentCrimesPerPop_pred'] < 0]
predictions_below_0[['absolute_error', 'ViolentCrimesPerPop_pred',
'ViolentCrimesPerPop']].head()

# %%
# Plot a boxplot of the error against race
sns.boxplot(x='RaceBlackBin5', y='absolute_error', data=full_dataset)
plt.xlabel('Race Black Bin 3')
plt.ylabel('Absolute Error')
plt.title('Absolute Error by Race Black Bin 3')
plt.show()

# %%
```

```
# Plot error against poverty level

sns.boxplot(x='PovBin5', y='absolute_error', data=full_dataset)

plt.xlabel('Poverty Level')

plt.ylabel('Absolute Error')

plt.title('Absolute Error by Poverty Level')

plt.show()



# %% [markdown]

# ## Markov Decision Process

#

# ### Initial Formulation

#

# An MDP with 6 states and 3 actions is formulated. Transition probabilities and reward values are created in the format required for `mdptoolbox`.


# %%

# Calculate the probability of each state in the dataset


print(f'Probability of each state in the dataset:')

print(f'\nPoverty Bin 3: {full_dataset["PovBin3"].value_counts(sort=False, normalize=True)}')

print(f'\nCrime Bin 3: {full_dataset["CrimeBin3"].value_counts(sort=False, normalize=True)}')




# %%

# Prepare transition probabilities (T) and rewards (R) for mdptoolbox
```

```
# States
states = [
    "High Crime, High Poverty",     # S0
    "Medium Crime, High Poverty",   # S1
    "Low Crime, High Poverty",      # S2
    "High Crime, Low Poverty",      # S3
    "Medium Crime, Low Poverty",    # S4
    "Low Crime, Low Poverty"        # S5
]

# Actions
actions = [
    "Increase Patrols",             # A0
    "Increase Community Policing",  # A1
    "Do Nothing"                    # A2
]

num_states = len(states)
num_actions = len(actions)

# Action-state-next_state transition matrix
T = np.zeros((num_actions, num_states, num_states))

# Reward matrix
R = np.zeros((num_states, num_actions))

# --- TRANSITIONS (T) ---
```

```
# --- HIGH POVERTY (Hard Mode) ---
# S0: High Crime. Sticky. Patrols struggle.
T[0, 0] = [0.8, 0.2, 0.0, 0, 0, 0] # Patrol
T[1, 0] = [0.7, 0.3, 0.0, 0, 0, 0] # Community
T[2, 0] = [1.0, 0.0, 0.0, 0, 0, 0] # Nothing

# S1: Med Crime. Volatile. Patrols cause backlash.
T[0, 1] = [0.3, 0.6, 0.1, 0, 0, 0] # Patrol (Risk of regression)
T[1, 1] = [0.1, 0.6, 0.3, 0, 0, 0] # Community
T[2, 1] = [0.5, 0.5, 0.0, 0, 0, 0] # Nothing (Fast regression)

# S2: Low Crime. Maintenance required.
T[0, 2] = [0.1, 0.1, 0.8, 0, 0, 0] # Patrol (Small %, big regression)
T[1, 2] = [0.0, 0.1, 0.9, 0, 0, 0] # Community
T[2, 2] = [0.0, 0.2, 0.8, 0, 0, 0] # Nothing

# --- LOW POVERTY (Easy Mode) ---
# S3: High Crime. Responsive to force.
T[0, 3] = [0, 0, 0, 0.4, 0.5, 0.1] # Patrol (Works well)
T[1, 3] = [0, 0, 0, 0.5, 0.4, 0.1] # Community
T[2, 3] = [0, 0, 0, 0.9, 0.1, 0.0] # Nothing

# S4: Med Crime. Resilient.
T[0, 4] = [0, 0, 0, 0.1, 0.3, 0.6] # Patrol
T[1, 4] = [0, 0, 0, 0.05, 0.45, 0.5] # Community
T[2, 4] = [0, 0, 0, 0.1, 0.9, 0.0] # Nothing
```

```
# S5: Low Crime. Stable.
T[0, 5] = [0, 0, 0, 0, 0.9, 0.1] # Patrol (Small regression risk)
T[1, 5] = [0, 0, 0, 0, 0.0, 1.0] # Community (Perfectly stable)
T[2, 5] = [0, 0, 0, 0, 0.05, 0.95] # Nothing (Small drift risk)

# --- REWARDS (R) ---
# R(s,a) = State Value - Action Cost
# State Values: High=-100, Med=0, Low=100
# Action Costs: Patrol=25, Community=10, Nothing=0

# High Poverty Rewards
R[0] = [-125, -110, -100]   # State 0 (High Crime)
R[1] = [-25, -10, 0]       # State 1 (Medium Crime)
R[2] = [75, 90, 100]       # State 2 (Low Crime)

# Low Poverty Rewards (Symmetric values)
R[3] = [-125, -110, -100]   # State 3 (High Crime)
R[4] = [-25, -10, 0]       # State 4 (Medium Crime)
R[5] = [75, 90, 100]       # State 5 (Low Crime)

R

# %% [markdown]
# ### Learning Strategy
#
# The Value Iteration strategy was used in the `mdptoolbox`. A myopic policy and patient policy were deployed to show diverse outcomes available.
```

```
# %%
# Myopic strategy
myopic_model = mdp.ValueIteration(T, R, discount=0.05)
myopic_model.run()
print('Myopic model policy:')
print(myopic_model.policy)

# Patient strategy
patient_model = mdp.ValueIteration(T, R, discount=0.95)
patient_model.run()
print('\nPatient model policy:')
print(patient_model.policy)


# %% [markdown]
# ## Appendices

# %% [markdown]
# ### Appendix 1

# %%
# Describes where the data comes from
dataset_abstract = metadata['abstract']
print(f'Abstract: {dataset_abstract}\n')

# Describes what the data contains, how it was normalised and any limitations
dataset_description = metadata['additional_info']['summary']
print(f'Description:\n {dataset_description}')
```

```
# %% [markdown]

# ### Appendix 2


# %%

# Almost all of the variables with extensive missing values were from the 1990 US LEMAS survey

# View the full set of variables from this survey

print('Table 19:\n')

variables_descriptions_enhanced[variables_descriptions_enhanced['source'] == '1990 US LEMAS']


# %% [markdown]

# ### Appendix 3


# %%

### This was a vast grid search over nearly 4,000 combinations for experimentation

### Commented out for smooth running of the notebook, the results were saved in a csv file (available on request) and outputs are shown below


'''import time


# Grid search across learning rates and batch sizes

# Batch size = len(X_train) tests batch gradient descent; smaller values test mini-batch gradient descent

print(f"{'LR':<8} | {'Batch Size':<12} | {'Val MSE':<6} | {'Epochs Run':<10} | {'Alpha':<10} | {'Momentum':<10} | {'Patience':<10} | {'Decay':<10} | {'Training Time':<10}")

print("-" * 100)
```

```
best_params = {}
best_history = None
min_error = float('inf')

results_list = []
all_histories = []  # Store all histories for plotting



for lr in [0.001, 0.01, 0.05, 0.1]:
    # Include batch gradient descent (full dataset) and mini-batch sizes
    batch_sizes = [32, 64, 128, len(X_train)]

    for bs in batch_sizes:

        for alpha in [0, 0.01, 0.1]:

            for momentum in [0, 0.2, 0.5, 0.7, 1]:

                for patience in [1, 5, 10, 50]:

                    for decay in [0.5, 0.7, 0.9, 1]:

                        np.random.seed(40466218)
                        theta_init = np.random.randn(X_train.shape[1])



                        start_time = time.time()
                        best_theta, history = train_linear_regression(
```

```
                X_train, y_train, X_val, y_val, theta_init, lr, 5000, bs,
momentum=momentum, patience=patience, decay=decay, alpha=alpha


            )


            end_time = time.time()
            training_time = end_time - start_time


            bs_display = 'Full' if bs == len(X_train) else bs
            print(f"{lr:<8} | {bs_display:<12} | {min(history['val_mse']):<.4f} |
{len(history['val_mse']):<4} | {alpha:<10} | {momentum:<10} | {patience:<10} |
{decay:<10} | {training_time:.2f} seconds")


            results_list.append({
                'Learning Rate': lr,
                'Batch Size': bs_display,
                'Val MSE': min(history['val_mse']),
                'Epochs Run': len(history['val_mse']),
                'Alpha': alpha,
                'Momentum': momentum,
                'Patience': patience,
                'Decay': decay,
                'Training Time': training_time
            })


            all_histories.append({
                'learning_rate': lr,
                'batch_size': bs_display,
                'history': history,
```

```
                'Alpha': alpha,
                'Momentum': momentum,
                'Patience': patience,
                'Decay': decay,
                'Training Time': training_time
            })

            best_val_mse = min(history['val_mse'])
            epochs_run = len(history['val_mse'])

            if best_val_mse < min_error:
                min_error = best_val_mse
                best_params = {'lr': lr, 'batch_size': bs}
                best_history = history


print(f"\nBest Params: {best_params}")

results_df.to_csv('expansive_grid_search.csv', index=False)'''



# %%
expansive_grid_search = pd.read_csv('expansive_grid_search.csv')
ranked_results = expansive_grid_search.sort_values(by='Val MSE', ascending=True)

ranked_results.head(20)
```

```
# %%
# Alpha of 0.1 is best

sns.barplot(
   data=expansive_grid_search,
   x='Alpha',
   y='Val MSE',
)



# %%
# Decay of 0.9 is best overall

sns.barplot(
   data=expansive_grid_search,
   x='Decay',
   y='Val MSE'
)

# %%
# Momentum declines gradually to peak at 0.7 before overshooting at 1.0

sns.barplot(
   data=expansive_grid_search,
   x='Momentum',
   y='Val MSE',
)
```

```
# %%
# At the lowest learning rate, full batch doesn't converge
# At the highest learning rate, 32 batch has high errors

sns.barplot(
    data=expansive_grid_search,
    x='Batch Size',
    y='Val MSE',
    hue='Learning Rate',
)

# %% [markdown]
# ### Appendix 4

# %%
def drop_high_vif_features(df, threshold=10):
    # Work on a copy so we don't destroy the original immediately
    df_vif = df.copy()

    # Keep looping until the maximum VIF is below the threshold
    while True:
        # 1. Add a constant (intercept) for the calculation
        # (We define this inside the loop because the feature set changes every time)
        df_with_const = add_constant(df_vif)

        # 2. Calculate VIF for every feature
        vif_series = pd.Series(
```

```
        [variance_inflation_factor(df_with_const.values, i)
         for i in range(df_with_const.shape[1])],
        index=df_with_const.columns
    )

    # 3. Drop the 'const' index so we don't remove the intercept
    if 'const' in vif_series.index:
        vif_series = vif_series.drop('const')

    # 4. Find the max VIF
    max_vif = vif_series.max()
    max_feature = vif_series.idxmax()

    # 5. Check threshold
    if max_vif > threshold:
        print(f"Dropping '{max_feature}' with VIF: {max_vif:.1f}")
        df_vif = df_vif.drop(columns=[max_feature])
    else:
        print(f"\nAll remaining features have VIF < {threshold}. Max is {max_vif:.1f}")
        break

  return df_vif

'''# Run the robust cleanup
# Make sure to use your numeric features only
X_numeric = X_final.select_dtypes(include=[np.number])
X_cleaned = drop_high_vif_features(X_numeric, threshold=10)
```

```
print(f"\nOriginal feature count: {X_numeric.shape[1]}")

print(f"Remaining feature count: {X_cleaned.shape[1]}")'''


kept_features = ['householdsize', 'racepctblack', 'racePctAsian', 'racePctHisp',
'agePct12t21', 'agePct65up', 'pctUrban', 'pctWFarmSelf', 'pctWInvInc', 'pctWPubAsst',
'pctWRetire', 'blackPerCap', 'indianPerCap', 'AsianPerCap', 'OtherPerCap', 'HispPerCap',
'PctLess9thGrade', 'PctUnemployed', 'PctEmplManu', 'PctEmplProfServ',
'PctOccupManu', 'MalePctDivorce', 'MalePctNevMarr', 'PctYoungKids2Par', 'PctTeen2Par',
'PctWorkMomYoungKids', 'PctWorkMom', 'NumIlleg', 'NumImmig', 'PctImmigRecent',
'PctImmigRec10', 'PctRecentImmig', 'PctLargHouseFam', 'PersPerRentOccHous',
'PctHousLess3BR', 'MedNumBR', 'HousVacant', 'PctHousOccup', 'PctHousOwnOcc',
'PctVacantBoarded', 'PctVacMore6Mos', 'MedYrHousBuilt', 'PctHousNoPhone',
'PctWOFullPlumb', 'OwnOccHiQuart', 'RentLowQ', 'MedRentPctHousInc',
'MedOwnCostPctInc', 'MedOwnCostPctIncNoMtg', 'NumInShelters', 'NumStreet',
'PctBornSameState', 'PctSameCity85', 'PctSameState85', 'LandArea', 'PopDens',
'PctUsePubTrans']


# %%

# Convert X_train/X_test to DataFrame if they are not already, to support column
selection by name

if not isinstance(X_train, pd.DataFrame):

   X_train_df = pd.DataFrame(X_train, columns=X_final.columns)

else:

   X_train_df = X_train.copy()

if not isinstance(X_test, pd.DataFrame):

   X_test_df = pd.DataFrame(X_test, columns=X_final.columns)

else:

   X_test_df = X_test.copy()


# Reset index to get default integer index for safe row shuffling

X_train_df = X_train_df.reset_index(drop=True)

X_test_df = X_test_df.reset_index(drop=True)
```

```
if hasattr(y_train, 'reset_index'):
    y_train_reset = y_train.reset_index(drop=True)
else:
    # If y_train is a NumPy array
    y_train_reset = np.asarray(y_train)

X_train_clean = X_train_df[kept_features]
X_test_clean = X_test_df[kept_features]

# Train the model on the clean data
clean_theta, clean_history = train_linear_regression(
    X_train_clean.values, y_train_reset, X_train_clean.values, y_train_reset,
    np.random.randn(X_train_clean.shape[1]), # New random weights for new shape
    best_lr, 1500, best_batch_size
)

mse_clean = lin_mse(y_test, lin_reg_predict(X_test_clean, clean_theta))
print(f"MSE of cleaned model: {mse_clean:.3f}")


# %%
# Show the clened model consistently underfitting

plot_predictions(y_test, lin_reg_predict(X_test_clean, clean_theta), "Cleaned Model Predictions vs True Values")


# %% [markdown]
```

Appendix 5

%%

Demonstrate the normal equation fails due to singularity

'''X_b_test = add_bias(X_test)

theta_best = np.linalg.inv(X_b_test.T @ X_b_test) @ X_b_test.T @ y_test'''

%% [markdown]

#

Notes on AI Use

#

#AI was used in two supporting pieces of the notebook:

- Plotting the nonlinear relationships between crime and poverty using regplot with annotations

- Calculating the variance influence factor

These were designed to semantically enhance and contextualise the dataset and interpretation, but did not play a significant role in the core analysis or model development. The majority of the code is boilerplate pandas, seaborn and matplotlib, and iterations of what we have practiced in class, so there was little need / opportunity for AI-assisted code generation.

%% [markdown]

#